PROJET SCIENTIFIQUE COLLECTIF - RAPPORT FINAL

Time series pattern classification for market prediction

25 avril 2022

Coordinateur : Julien Combe, **Tuteurs** : Fabrice Daniel, François De La Bourdonnaye
Elèves : Sacha Braun, Abdelhamid Jenzri, Florian Labaye, Mohamed Lakhnichi, Louis Lorrach, Meilame Tayebjee

TABLE DES MATIÈRES

Remerciements **3**

Introduction **4**
Motivation générale . . . 4
Vulgarisation de notre modèle . . . 4
État de l'art . . . 8

1 Preprocessing : La préparation des données **9**
1.1 Un premier traitement . . . 9
1.2 Utilisation des auto-encodeurs . . . 9
1.2.1 Qu'est ce qu'un auto-encodeur ? . . . 9
1.2.2 Pourquoi nous intéressent-ils ? . . . 11
1.2.3 Les deux grandes problématiques mises en évidence . . . 12
1.2.4 Implémentation et résultats . . . 14
1.3 Perspectives d'amélioration . . . 16

2 Clustering **18**
2.1 La classification des données : l'algorithme KMeans . . . 18
2.2 Création d'un indicateur de dispersion . . . 20
2.3 Intersecter des sets de clusters . . . 20
2.4 La sélection de clusters . . . 21
2.5 L'innovation et sa paramétrisation . . . 22
2.5.1 Le nombre de clusters . . . 23
2.5.2 Le seuil de tolérance des intersections . . . 23
2.6 Le nombre de raffinements . . . 24
2.7 Les résultats obtenus . . . 25
2.8 Conclusion et perspectives . . . 26

3 Prédiction et Backtesting **27**
3.1 Prédiction . . . 27
3.1.1 Principe . . . 27
3.2 Backtesting sur le marché . . . 29
3.2.1 Backtesting sur l'indice FRA40 . . . 30

4 Conclusion **35**

Appendix **37**

Bibliographie **44**

REMERCIEMENTS

Nous tenons d'abord à remercier chaleureusement Julien COMBE, notre coordinateur, pour la proposition de ce sujet, sa disponibilité et son aide dans le cadrage des attentes.

Nous remercions nos partenaires de LUSIS, sans qui ce projet n'aurait pu être mené à bout. Merci à Fabrice DANIEL pour la formation en finance de marché qu'il nous a dispensée, et pour l'indication de ressources utiles en intelligence artificielle. Nous remercions grandement François De La BOURDONNAYE pour son suivi hebdomadaire, son aide dans le guidage de nos recherches, et les nombreuses solutions qu'il nous a apportées et indiquées. Son aide a été capitale.

Enfin, nous remercions l'Ecole polytechnique, et en particulier le département d'économie, dont le support logistique a permis l'établissement de ce travail.

INTRODUCTION

Motivation générale

35%. C'est le rendement annuel moyen du fond d'investissement Renaissance depuis sa création en 1988. Fondé par le mathématicien James Simon, ce fond utilise des techniques quantitatives et computationnelles avancés pour faire de la négociation algorithmique. La machine remplace l'humain pour prédire les évolutions des cours de bourse.

Pourtant, la théorie financière traditionnelle a toujours considéré les marchés financiers comme efficients : le prix des actifs financiers représentent parfaitement toute l'information disponible et la chance devient la seule manière de gagner de l'argent. Mais si les mouvements des actifs boursiers sont totalement aléatoires comment est-il possible que les gestionnaires de fonds de trading à haute fréquence soient parmi les professionnels les mieux payés au monde ? C'est bien que les prix des actifs financiers ne sont pas totalement régis par le hasard. Dès lors les machines s'emparent des marchés financier, à la recherche de nouvelles stratégies gagnantes pour générer des profits. En 2013 sur le marché américain, on évaluait à plus de 80% la part des transactions boursières réalisées par des robots [11]. Alors c'est à notre tour de rechercher à prédire le marché en nous intéréssant à l'analyse des time series, des séquences de prix.

Vulgarisation de notre modèle

Ce projet examine l'utilisation des algorithmes et des techniques d'apprentissage automatique pour prédire l'évolution des cours des actifs financiers à très court terme (par exemple, de quelques minutes à quelques heures). L'objectif à la fin est de concevoir un système décisionnel capable de prédire la tendance future du prix, en se basant sur les flux de données des transactions antérieures. Le système utilisé est celui des time series : nous classons les séries temporelles des évolutions du cours des cours boursiers pour identifier des séquences prédictives. Nous avons proposé trois étapes clés dans notre travail : atténuer le bruit des données, classer les time series, et identifier les time series prédictives.

Ce travail de recherche a déjà été entammé par un groupe l'année précédent la notre. Notre projet s'inscrit dans un fil rouge pour que chaque année les groupes apportent leurs améliorations. Ainsi notre objectif a été de travailler dans la continuité du travail déjà réalisé et non pas de refaire ce qui avait été réalisé. Le cahier des charges présenté par Lusis était clair : améliorer la stabilité du modèle créé l'année dernière tout en conservant une statégie gagnante.

Cette introduction vulgarise notre modèle afin de comprendre les termes techniques que nous utiliserons, ainsi que chaque étape clé de notre projet. Cette partie est essentielle à la

lecture dans la mesure ou nous présentons surtout le travail effectué l'année dernière pour ensuite comprendre les analyses techniques et spécifiques que nous effectuons notre année. Pour en simplifier la compréhension, nous utiliserons volontairement des exemples caricaturaux et des courbes simples, ce qui n'est évidemment pas le cas de la réalité.

Prenons par exemple la courbe d'un indice financier. Notre objectif est de retrouver des patterns, des séquences, que nous pourrons ensuite classer comme prédictives, ce que nous allons expliquer ici. Nous commençons donc par segmenter cette courbe en courbes de longueur L, pour étudier précisement des séquences. C'est ce que nous faisons ici en figure 1 et 2 :

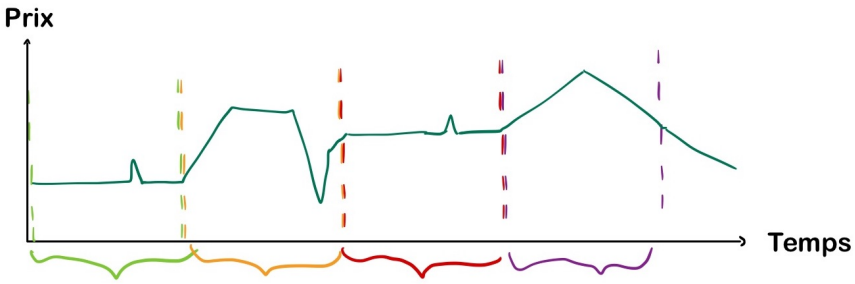


Figure 1 – Cours initaux à étudier

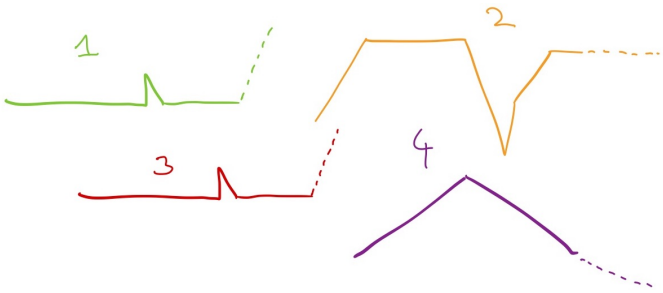


Figure 2 – Courbes de longueur L à classer

Sur le second graphique, nous avons retranscrit les différentes courbes extraites de la courbe des marchés financiers. En couleur pleine figure la séquence à étudier, et en pointillés les cours du marché qui suivent directement la séquence à étudier. Il est facile de conclure que dans notre exemple, suite à un motif de la forme :

les cours de l'indice étudié augmentent (voir les cours en pointillés figure 2). Une idée serait donc d'investir dès que nous retrouvons une fluctuation similiaire à cette séquence dans le futur.

Mais s'il est simple de faire cette conclusion à l'oeil nu, il faut que l'ordinateur soit capable de faire la même conclusion. C'est la partie clustering. L'idée est de classer chacunes de ces time series dans des clusters : dire à l'ordinateur que certaines courbes sont similaires et les classer dans le même "tiroir". Nous avons utilisé l'algorithme de KMeans qui sera expliqué en 2.1. L'idée est d'indiquer à l'ordinateur le nombre de catégories, de "tiroirs" que nous voulons. Par exemple ici si nous indiquons vouloir deux catégories, l'algorithme va classer ces quatre courbes en deux catégories. Par exemple, dire que les courbes (2) et (4) se ressemblent, de même que pour les courbes (1) et (3). Nous avons alors des clusters qui correspondent aux différentes catégories de courbes.

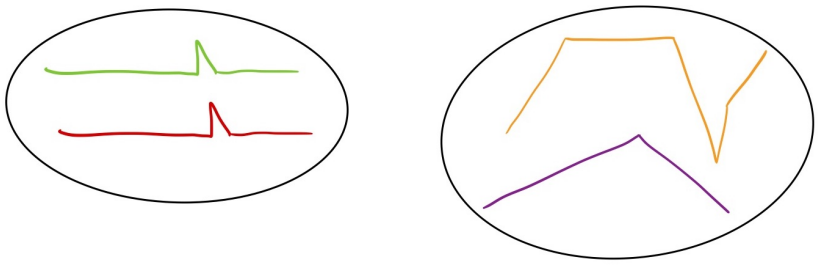


Figure 3 – Deux clusters qu'il est possible d'obtenir dans notre cas

Une fois que nous avons classé nos courbes, nous pouvons travailler sur les clusters : on entraîne les données. C'est l'étape de la filtration : dire pour chaque cluster s'il a des chances de prédire une hausse ou une baisse dans le futur. Ici, nous pouvons conclure que le cluster 1 prédit une hausse du marché : il y a une hausse des cours de l'indice entre le temps initial et le temps final dans chaque séquence dans ce cluster, tandis que le cluster 2 est plus indécis : il y a une baisse et une stagnation des cours suite à ces courbes.

Cela nous permet d'effectuer la dernière étape : celle du backtesting. Nous testons notre modèle avec des données non entrainées pour observer s'il est efficace ou non. Ici, nous pourrions établir cette stratégie pour le futur par exemple :

1. Nous sélectionnons la séquence de longueur L qui se termine à la date actuelle pour l'étudier.
2. Nous la comparons au set de clusters : si nous devions classer cette courbe, dans quel cluster déjà crée l'ajouter ? À quelles courbes ressemble cette séquence ?
3. Si nous la mettons dans le cluster 1 : nous choisissons d'investir en prédisant une hausse des marchés.
4. Si nous la mettons dans le cluster 2 : nous choisissons de ne rien faire car les marchés sont trop incertains.

Ce raisonnement élémentaire est un exemple de vulgarisation de la base de notre travail, et de ce qui a été effectué l'année dernière.

Revenons plus en détails sur certains points, notamment les points sur lesquels nous avons travaillé cette année. Tout d'abord, nous avons triché dès le départ en fournissant une belle courbe en figure 1. En réalité, les cours de la bourse sont beaucoup plus bruités. Il devient alors très compliqué pour l'ordinateur de classer ces séquences car le bruit complexifie les calculs et conduit à de l'overfitting (l'ordinateur veut tellement bien faire qu'il interprète trop les détails et passe à côté d'une tendance général). Pour cette raison, nous devons lisser nos courbes, réduire le bruit présent dans les données. C'est le travail de l'auto-encodeur. D'une part l'auto-encodeur lisse les courbes, mais d'autre part il réduit les dimensions de celles-ci : si nous travaillons avec des séquences de 8 points par exemple, l'auto-encodeur va réduire cette dimension pour simplifier les calculs. Cette particularité sera plus expliquée dans la partie 1.

Un problème des auto-encodeurs est leur manque de stabilité : si nous faisons tourner un algorithme pour encoder des séquences deux fois de suite, nous aurons deux résultats différents. Une problématique de notre projet est donc de travailler sur cette stabilité. C'est le sujet que nous aborderons dans notre première partie. Comment travailler sur des algorithmes d'auto-encodeurs pour gagner en efficacité ?

Une fois que nous avons encodé les données, nous repassons donc à l'étape du clustering. Dans notre exemple nous avons demandé 2 clusters, mais pourquoi en avoir demandé 2 plutôt que 3 ? Nous aurions très bien pu nous retrouver avec la classification qui semble même plus adéquate en figure 4.

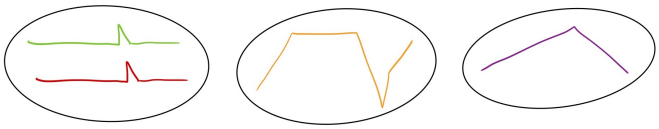


Figure 4 – 3 Clusters qu'il est possible d'obtenir

Comment trouver le nombre de clusters nécessaires quand nous avons des dizaines de milliers d'informations ? Nous répondrons à cette question dans la seconde partie du projet qui portera sur le clustering. Mais là encore, l'algorithme de clustering n'est pas stable. En le faisant tourner plusieurs fois, nous obtenons des résultats différents. Par exemple ici, nous aurions très bien pu avoir les courbes (1) (2) et (3) dans la même catégorie, et la courbe (4) seule en refaisant tourner l'algorithme. Mais alors la stratégie aurait été différente : cette fois-ci, nous aurions shorté (on gagne de l'argent si les cours baissent) si notre séquence ressemble à la séquence du cluster 2. Nous comprennons alors bien pourquoi nous recherchons à avoir de la stabilité dans nos algorithmes : pour qu'une stratégie soit gagnante, il faut qu'elle fonctionne qu'importe les données renvoyées par l'algorithme. Si notre statégie est gagnante une première fois puis qu'en refaisant tourner l'algorithme elle est perdante, alors il est impossible de conclure sur l'efficacité de notre statégie. Pour cette raison nous avons également travaillé sur la stabi-

lité de nos algorithmes de clustering. Comment obtenirs des sets de clusters plus stables pour travailler dessus ? Nous présenterons nos recherches et leurs conclusions dans une seconde partie.

Une fois ces clusters obtenus, il faut travailler sur chacuns d'entre eux pour simuler des stratégies de marché. Comment définir si un cluster a des chances de prédire une hausse ou une baisse de marché ? Nous travaillerons sur cela dans la dernière partie de notre projet. Enfin, nous pourrons tester notre modèle : est-il gagnant ? est-il stable ? Nous rappelons en effet le cahier des charges de LUSIS : améliorer le modèle crée l'année dernière pour gagner en stabilité tout en conservant un modèle gagnant.

État de l'art

Avec l'émergence de l'ère du *Big Data* et la disponibilité de quantités massives de données, les algorithmes d'apprentissage profond (*Deep Learning*) ont de nouveau été mis à l'honneur. Parmi ceux-ci, les auto-encodeurs ont de plus en plus été utilisés depuis quelques années pour la compression, le débruitage et la reconstruction de données lacunaires. Le processus d'apprentissage profond permet de jouer à plein sur la massive quantité de données disponibles [14].

Plus encore, certains type d'auto-encodeurs, particulièrement les LSTM, sont, par leur structure, idéaux dans le cadre de problématiques de *Times Series*. De nombreux travaux (récents) mettent en évidence l'efficacité de ces outils [15] [16].

Les algorithmes de clustering ont d'abord été utilisés pour reproduire ou améliorer les performances d'un index boursier en utilisant une partie ou tous les actifs qu'il contient à des ratios différents. [12]

De nombreuses stratégies de clustering de Time Series sont developpées dans le domaine de la santé. Les courbes à classer sont alors très différentes et l'utilisation d'une distance non euclidienne, la Dynamic Time Wrapping, est très efficace. [13]

Quant au backtesting, il reste une composante indispensable dans tout travail de modélisation et d'étude statistique et surtout en finance. Les usages du backtesting dans la litterature sont divisés en deux catégories ; un usage qui concerne l'évaluation des risques liés à la gestion des portefeuilles [9], et un autre usage qui sert à évaluer les stratégies de trading [10].

1 PREPROCESSING : LA PRÉPARATION DES DONNÉES

1.1 Un premier traitement

Nous avons reçu l'accès à la base de données Lusis. Cette base de données comporte différents indices (CAC40, SP500. . .), du FOREX (EUR/USD, EUR/GBP, ...) et des matières premières (OIL, CUIVRE. . . .).

Pour chaque type, nous avons les valeurs OPEN,HIGH, LOW, CLOSE, VOLUME **toutes les 5 minutes** sur des périodes de temps différentes.

Nous avons choisi, dans un premier temps, de nous concentrer sur le CAC40 (indice FRA40), et de travailler sur des séquences (*pattern*) **des valeurs OPEN** d'une durée de 4 heures, soit des vecteurs de $\mathbf{R}^{48}$. Pour réduire la dimension de ces vecteurs, avoir des courbes plus lisibles et moins accidentées, et parce qu'il est possible d'allonger le pas de temps sans perdre d'information sur la tendance des *patterns*, nous n'avons gardé que 8 points sur les 48, soit les valeurs de OPEN toutes les 40 minutes, qui font une séquence d'une durée de 4H.

Le clustering portera donc sur ces quelques dizaines de milliers de séquences, une séquence étant donc un vecteur de $\mathbf{R}^{8}$.

1.2 Utilisation des auto-encodeurs

1.2.1 • Qu'est ce qu'un auto-encodeur ?

Un auto-encodeur est un algorithme d'apprentissage **non supervisé** [1], basé sur des réseaux de neurones, qui cherche à apprendre une version compresssée d'une entrée, puis être capable de la reconstruire avec plus ou moins d'erreurs. [1]

Un auto-encodeur est composé de deux parties : **l'encodeur** et **le décodeur**.

L'encodeur est une couche de neurones (la plupart des structures sont déjà implémentées dans la bibliothèque Keras sur Python : les plus connues sont LSTM (*Long Short Term Memory*) et GRU (*Gated Recurrent Units*)) qui ne garde que les informations "essentielles" de l'entrée, lui extrait sa moelle épinière, pour produire en sortie des vecteurs d'une dimension réduite, qu'on appellera *dimension latente* ou *dimension de l'espace latent*. **Cette dimension latente est choisie par l'utilisateur**.

1. Non supervisé signifie que l'utilisateur ne "labellise" pas les données d'entrée : c'est l'algorithme qui trouve automatiquement les critères (*features*) d'importance

On appelle l'ensemble des vecteurs d'entrée une fois compressés *l'espace latent*. A noter pour la suite que, mise à part la dimension de cet espace que nous fixons effectivement, l'utilisateur n'a aucun autre contrôle *a priori* sur cet espace, il est le pur produit de phénomènes stochastiques - c'est ce qui est appelé le phénomène *black box* (boîte noire) en deep learning.

Le choix de la dimension latente repose sur un arbitrage : on veut à la fois compresser les données d'entrée le plus possible, donc choisir la plus petite dimension latente ; sans pour autant perdre trop d'informations, faute de quoi on perdrait l'essence des données originelles.

La deuxième partie de l'autoencodeur, le décodeur, prend en entrée des vecteurs de l'espace latent, donc des données d'entrée ayant été compressées, pour les re-traduire dans l'espace de départ, les **décompresser**.

Si bien que grâce à l'architecture une fois construite de l'auto-encodeur, nous avons accès à :

1. des vecteurs *compressés* de dimension latente choisie par nous-mêmes, qui contiennent les informations essentielles de nos séquences d'entrée ; nous pouvons travailler sur ces vecteurs compressés, typiquement faire du clustering...
2. Une fois que nous avons "travaillé" sur ces vecteurs compressés, nous pouvons les décoder pour rebasculer dans l'espace lisible, compréhensible, que nous connaissons : l'espace de départ avec des séquences de 8 points couvrant une période de 4h.

Le schéma suivant (figure 5) résume la structure d'un auto-encodeur.

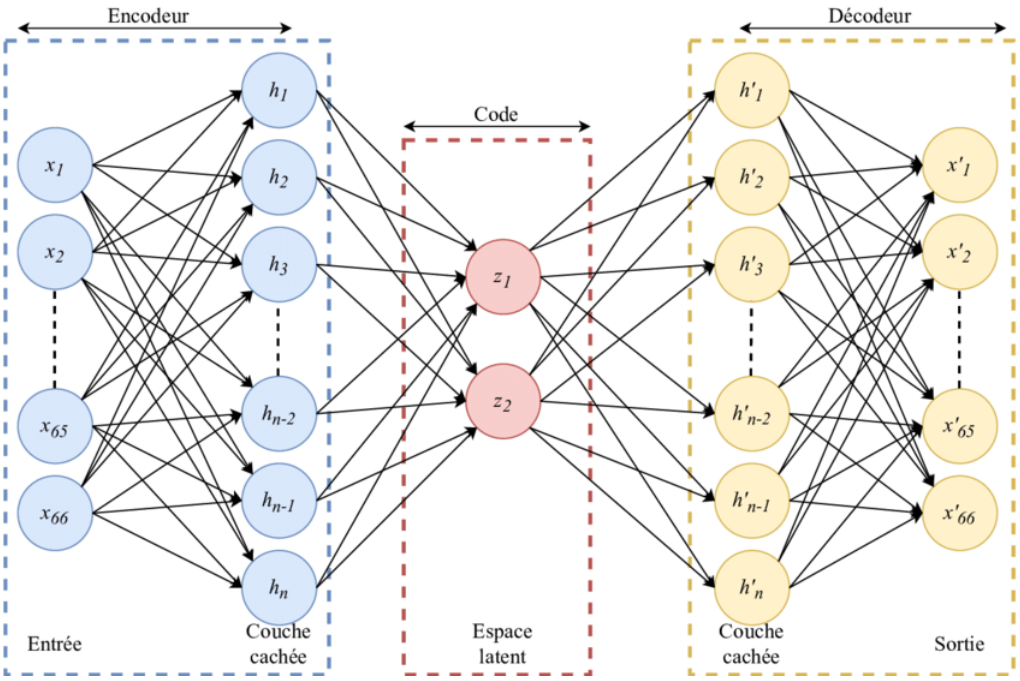


Figure 5 – Structure d'un auto-encodeur

1.2.2 • Pourquoi nous intéressent-ils ?

Les auto-encodeurs ont un double intérêt pour nous.

D'une part, la réduction de la dimensionnalité de nos vecteurs d'entrée est clé pour ce sujet. En effet, les algorithmes de clustering, en particulier le plus classique d'entre eux : *KMeans*, sont extrêmement gloutons en puissance de calcul, si bien que faire du clustering sur des séquences de 48 points était inenvisageable au vu de la taille de la base de données fournie par LUSIS. Nous avons déjà réduit à 8 la dimension de nos séquences d'entrée par le premier *preprocessing*, mais les auto-encodeurs vont nous permettre d'avancer encore plus loin, en choisissant une dimension latente entre 2 et 4. Le clustering de l'étape d'après pourra donc s'effectuer non plus sur des vecteurs de $\mathbf{R}^8$, mais des vecteurs (d'au plus) $\mathbf{R}^4$ qui seront en sortie de *l'encodeur* : nous construirons en effet notre auto-encodeur de façon à pouvoir isoler la sortie de l'encodeur (les vecteurs latents). Une fois le clustering effectué, on pourra utiliser la partie décodeur pour visualiser les clusters dans leur espace de départ.

D'autre part, le processus encodage-décodage de l'autoencodeur permet de se concentrer uniquement sur les points saillants des vecteurs d'entrée ; l'algorithme d'apprentissage non supervisé permet d'identifier automatiquement les caractéristiques les plus importantes, celles les plus porteuses d'informations. Dans notre situation, les séquences de points vont se voir *lissées*, *débruitées*, en sortie de l'auto-encodeur, qui au moment de la reconstruction se passera des détails et des "pics" qu'il considèrera non pertinents. Cela permet d'avoir des séquences de meilleure lisibilité, ce qui est d'une grande utilité pour avoir un clustering plus efficace qui ne se laisse plus brouiller par les bruits qu'il y avait dans chaque séquence initialement, où l'on identifie mieux la tendance générale (haussière ou baissière), ce qui est très utile dans la partie *backtesting*, où l'algorithme aura plus de capacité de prédiction grâce à ces clusters *lissés*.

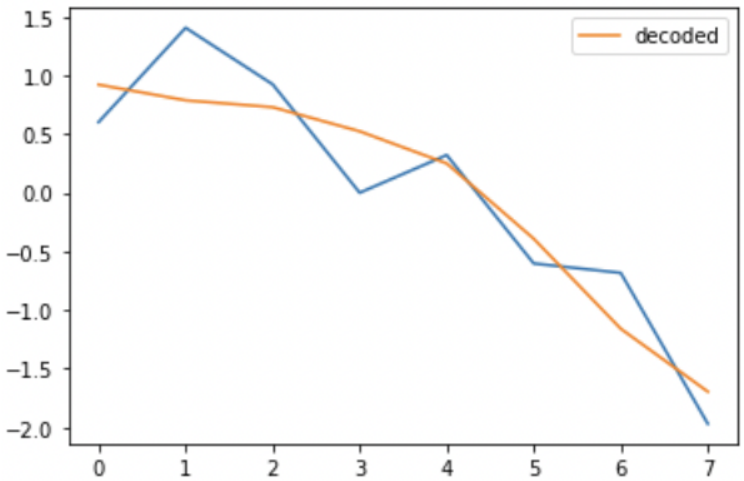


Figure 6 – Une séquence initiale en bleu et la séquence décodée après passage dans l'auto-encodeur LSTM en orange, dimension latente choisie de 2

1.2.3 • Les deux grandes problématiques mises en évidence

A noter que dans le cadre de notre projet, et avec concertation avec notre tuteur, nous nous sommes concentrés sur des auto-encodeurs de type LSTM. Ceux-ci sont tout particulièrement adaptés aux problèmes de type *Times Series* (séries temporelles, ce qui correspond à nos inputs ici), car sa structure lui permet de garder en mémoire les valeurs passées sur un intervalle de temps donné ; d'où le nom *Long Short Term Memory.* Des détails techniques sur la structure de ces auto-encodeurs et leurs différentes applications (reconnaissance manuscrite, reconnaissance auditive...) seront donnés en bibliographie. [2][3][4]

Erreur de reconstruction

Concernant les auto-encodeurs, la principale problématique reste celle de **l'erreur de reconstruction**, c'est-à-dire l'écart entre les données d'entrée et les données décodées. Bien entendu, un écart dû au débruitage est naturel, sain et souhaité : la structure même de l'auto-encodeur fait que cet écart ne peut disparaître et persiste quelle que soit l'action de l'utilisateur. En revanche, certains écarts sont à préserver afin d'éviter que les séquences décodées n'aient plus rien à voir en termes de tendance haussière ou baissière avec les séquences de départ : de telles imperfections, on l'a vu, auraient des impacts dans la partie *backtesting* du projet.

Dès lors, notre auto-encodeur doit produire des courbes décodées qui soient certes lissées, débruitées (elles le seront dans tous les cas grâce au processus compression-décompression), mais qui doivent tout de même garder l'information des *tendances* : c'est par exemple le cas dans la figure 6.

Problématique de stabilité

L'autre grande problématique d'importance a été spécifiquement fixée par LUSIS dans le cadre de notre projet : il s'agit de la **stabilité**, soit, en d'autres termes, le fait d'obtenir les mêmes résultats en relançant le code - l'objectif est de pouvoir appliquer nos algorithmes sur différents indices boursiers très rapidement et facilement, et de pouvoir basculer de l'un à l'autre.

Plus précisément, dans cette sous-partie dédiée aux auto-encodeurs, la principale source potentielle d'*instabilité* au sens entendu par LUSIS provient de cette boîte noire qu'est l'espace latent. Résultat de processus purement stochastiques (réseaux de neurones et descente de gradient), l'utilisateur n'a aucun contrôle dessus. Si bien que, en relançant deux fois d'affilée le même code, on peut potentiellement tomber sur des espaces latents complètement différents ; ce qui pose problème pour notre projet, puisque l'algorithme de clustering s'effectuera sur ces vecteurs de l'espace latent, qui peuvent changer du tout au tout. Cette instabilité de l'auto-encodeur peut se répercuter en une instabilité sur le clustering, qui *in fine* peut potentiellement changer complètement nos prédictions entre deux compilations du même code : nous cherchons à éviter cette situation.

Nous avons dans un premier temps commencé par confirmer cette intuition d'instabilité et essayé de mettre en place des indicateurs pour la révéler concrètement.

Pour mieux matérialiser cette instabilité et confirmer notre intuition, nous avons dessiné, pour une dimension latente fixée à 2 (ce qui nous permet de tracer ces points), l'intégralité des points de deux espaces latents produits en faisant tourner **le même code** deux fois d'affilée. La Figure 7 le montre : le résultat est sans appel, les deux espaces latents n'ont pas un seul point en commun, leur structure même est complètement différente : en bref, l'auto-encodeur est instable.

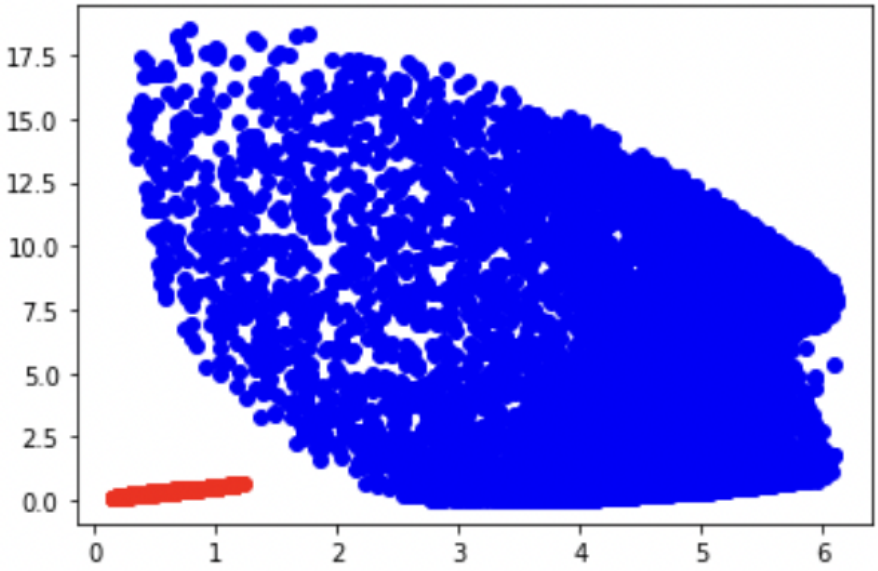


Figure 7 – Deux espaces latents (bleu et rouge) simulés d'affilée

Pour continuer sur cette problématique de la *mesure de l'instabilité*, et sur comment mettre en évidence l'instabilité de notre auto-encodeur, nous avons imaginé la notion de *centroïde* de l'espace latent, qui serait un vecteur contenant, pour chaque coordonnée, la moyenne de cette cooordonnée sur l'ensemble des vecteurs de l'espace.

Dans le cas d'un espace latent à deux dimensions : $centroide = (\frac{1}{n}\sum_{i=1}^{n} x_i, \frac{1}{n}\sum_{i=1}^{n} y_i)$.

Nous avons ensuite généré une dizaine d'espaces latents de dimension 2 (en faisant tourner le code dix fois d'affilée), et compilé leurs centroïdes. L'affichage de ces centroïdes (figure 8) montre une véritable instabilité de l'auto-encodeur et confirme l'intuition initiale. Les points sont très dispersés.

Entre plusieurs compilations du modèle, les espaces latents sont très dispersés et différents, dû au phénomène de *black box* des modèles d'apprentissage profond dont font partie les auto-encodeurs. Cette dispersion n'est pas un problème dans l'absolu, mais elle l'est particulièrement dans le cadre de notre projet et des exigences de LUSIS, où l'on a besoin d'espaces latents qui soient stables et *prévisibles*.

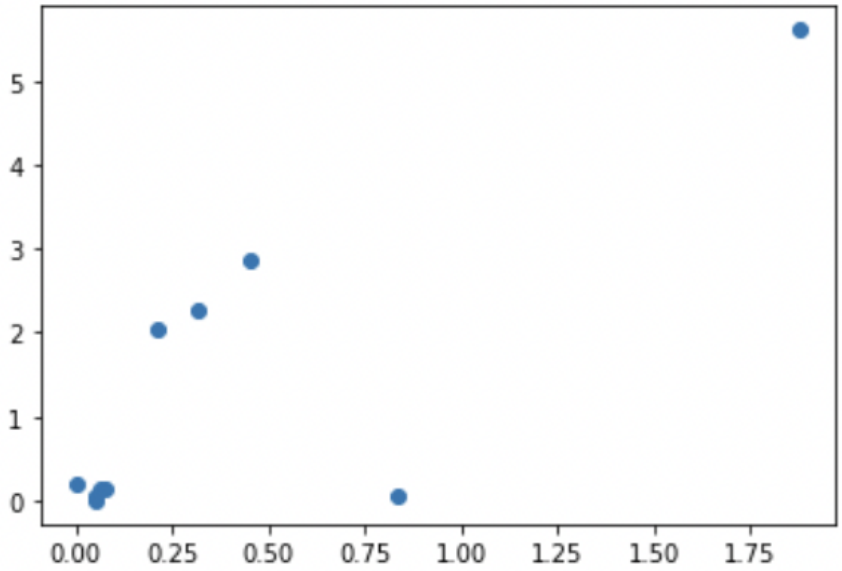


Figure 8 – Une dizaine de centroïdes placés dans le plan

1.2.4 • Implémentation et résultats

Optuna pour limiter l'erreur de reconstruction

Lorsque nous avons commencé à utiliser l'autoencodeur, nous choisissions des hyper-paramètres de manière aléatoire, en observant leurs effets sur la stabilité de l'autoencodeur et sur la fonction de perte. Mais cela ne nous permettait pas de choisir les meilleurs paramètres. C'est pourquoi nous avons utilisé Optuna, un cadre d'auto-optimisation d'hyper-paramètres.

Les hyper-paramètres sont des paramètres réglables qui permettent de contrôler le processus d'entrainement du modèle. Le réglage des hyper-paramètres est une étape cruciale en machine learning, car leur optimisation détermine le bon fonctionnement de l'algorithme d'apprentissage automatique.

Dans notre situation, les deux hyper-paramètres dont nous cherchons à connaître les meilleures valeurs, celles qui réduisent au mieux l'erreur de reconstruction (l'écart entre la séquence originelle et la séquence décodée), sont le nombre d'*epochs* et le *batch size*.

Un auto-encodeur, et plus généralement toute structure de réseaux de neurones, parcourt un certain nombre de fois l'ensemble des données d'entraînement pour mettre à jour les paramètres internes (appelés *poids*) de chaque neurone. Ce nombre de fois où la totalité des données d'entraînement est parcourue est le nombre d'*epochs*.

Le *batch size*, quant à lui, est (littéralement) la taille du paquet, le nombre d'exemples traités avant de mettre à jour les poids. En d'autres termes, si on choisit un nombre d'*epochs* de

100 avec un *batch size* de 20, on parcourt 100 fois les données d'entraînement par *paquets* de 20. Plus le *batch size* est élevé, plus on parcourt vite une *epoch*, mais on risque d'avoir un modèle peu précis ; plus le batch size est petit, plus on parcourt chaque donnée individuellement, donc plus on est exacts, mais il y a alors un risque d'*overfitting* (le modèle est trop collé aux données d'entraînement, donc inutilisable sur de nouvelles données).

Nous avons commencé par installer la bibliothèque Optuna. Pour réaliser l'optimisation, on définit la fonction objectif, c'est-à-dire la fonction qu'on veut optimiser. On choisit dans notre cas la fonction qui prend les deux hyperparamètres *epochs* et *batchsize* en entrée, et qui renvoie en sortie la distance entre les données avant traitement de l'autoencodeur, et les données après le décodage. Cette distance est donnée par la fonction de perte (*Mean Absolute Error*). L'utilisation d'Optuna nous permet alors de trouver les hyperparamètres qui minimisent cette fonction de perte, c'est-à-dire l'erreur qui découle de l'encodage et du décodage de l'auto-encodeur.

Grossièrement, Optuna teste la valeur de la fonction objectif pour plusieurs couples (epochs, batchsize), dans un intervalle spécifié par l'utilisateur, sans tester *tous* les couples - ce qui serait inefficace. Optuna sait dans "quelle" direction avancer, avant enfin de renvoyer la meilleure valeur des hyperparamètres. Plus de détails sur le fonctionnement technique d'Optuna sont donnés en bibliographie. [5][6]

Transformation en cos pour la stabilité

Une solution pour "tuer" complètement le problème de la stabilité a été de ne plus tout utiliser les auto-encodeurs, mais de s'en inspirer *via* la transformation en cos dite *DCT*.

Pour une séquence de 8 points $(x_k)_{0\leq k\leq 7}$, on appelle transformée en cos la séquence :

$$y_k = 2\sum_{n=0}^{7} x_n cos(\frac{\pi k(2n+1)}{16})$$

.

Manuellement, on fixe les $y_4 = ... = y_7 = 0$, si bien qu'on ne "garde" que les coefficients $y_0...y_3$: c'est ici que se trouve la réduction de dimensionnalité.

Pour décoder et rebasculer dans l'espace de dimension 8, on utilise la formule d'inversion, dite *IDCT* :

$$\forall k \in \{0...7\}\,, x_k^{décodé} = y_0 + 2\sum_{n=0}^{7} y_n cos(\frac{\pi n(2k+1)}{16})$$

Cette méthode étant déterministe (il n'y a absolument aucun phénomène stochastique), elle est parfaitement stable : en faisant tourner deux fois d'affilée le même code, on obtient exactement les mêmes résultats.

En revanche, la reconstruction, le *lissage* des courbes se fait avec moins de précision et plus d'erreurs, comme on peut le voir sur la figure 9 ; en fait cet algorithme n'utilise pas le côté massif de la base de données fournie par Lusis, tandis qu'un algorithme d'apprentissage

profond comme celui de l'auto-encodeur permet de jouer à plein sur la grande quantité de données (l'apprentissage profond précise ses paramètres en parcourant un grand nombre de fois les données d'entrée : plus celles-ci sont nombreuses, plus le modèle est redoutable) bien qu'il soit instable.

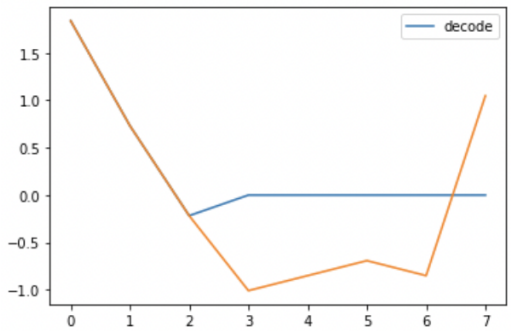


Figure 9 – Un exemple de courbe décodée (en bleu, courbe originale en orange) peu satisfaisante avec cette méthode.

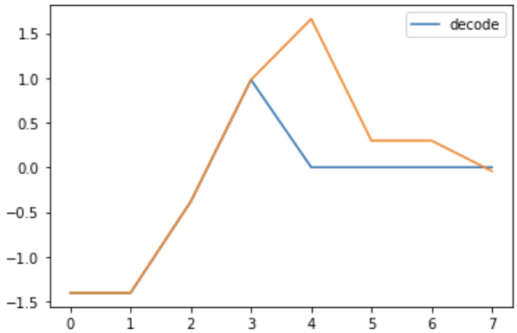


Figure 10 – Un exemple un peu meilleur avec la compression par DCT-IDCT

1.3 Perspectives d'amélioration

Afin de remédier à cette problématique d'instabilité de l'auto-encodeur tout en gardant les bienfaits de l'apprentissage profond, une des solutions que nous avions identifiées était de changer d'architecture et d'utiliser non pas des couches LSTM, mais des couches *variationnal LSTM*, qui en sont une variante. La grande différence avec les couches LSTM classiques est que les Variationnal LSTM posent des contraintes sur l'espace latent : leur structure est plus complexe, avec une composante de perte additionnelle, et un algorithme d'apprentissage spécifique. Les contraintes imposées sur les espaces latents "variationnels" nous laissent croire à une meilleure stabilité de ces auto-encodeurs. [7]

Les auto-encodeurs variationnels ne sont pas disponibles sur la bibliothèque Keras de Tensorflow. Avec l'aide de notre tuteur, nous avons essayé de le coder nous-mêmes, mais sans succès. Avec du temps supplémentaire, nous aurions approfondi la démarche d'implémentation, ainsi que la recherche théorique sur les contraintes concrètes posées par ces auto-encodeurs sur l'espace latent.

2 CLUSTERING

Position du problème

Un des objectifs fixés en coordination avec nos partenaires de Lusis concerne la stabilité des clusters obtenus par l'algorithme de clustering. En effet, en exécutant plusieurs fois le même algorithme de clustering, on se rend compte que la classification finale des données est aléatoire selon les exécutions. De plus, de façon quantitative, on se rend compte que ces différentes classifications sont éloignées les unes des autres. Concrètement, cela signifie que l'algorithme qui classifie les portions de courbe financière en collections de portions similaires a un facteur aléatoire : quand on le fait tourner plusieurs fois, les catégories de portions créées ne sont pas les mêmes. Cela représente un problème, dans la mesure où ces catégories obtenues sont assez différentes entre deux exécutions. Par conséquent, ce facteur aléatoire fragilise la capacité prédictive de notre mécanisme : on verra comment la stabilisation de cet algorithme permet d'améliorer l'efficacité de la prédiction.

L'enjeu a donc été d'imaginer une solution qui permette de stabiliser les sets de clusters obtenus (i.e. les classifications fournies par l'algorithme de clustering), pour assurer une pseudo-répétabilité de l'expérience. Il s'est agi ensuite d'implémenter informatiquement les pistes envisagées. Pour ce faire, nous avons procédé de façon autonome : dans la littérature informatique, peu d'articles font référence à ce problème, et nous avions à coeur de réfléchir de bout en bout à une solution qui nous était propre. Nous avons donc échangé et confronté les idées des membres du groupe, éliminant de nombreuses propositions, trop difficiles à implémenter ou pas assez rigoureuses, en conservant les plus pertinentes.

2.1 La classification des données : l'algorithme KMeans

Plusieurs algorithmes de machine learning permettent d'effectuer du clustering. En concertation avec Lusis et pour garder une cohérence avec le projet entamé l'année dernière, nous avons utilisé l'algorithme KMeans. L'année dernière, cet algorithme était utilisé avec ses imperfections, sans objectif d'optimisation : nous nous inscrivons dans la continuité du projet en optimisant cet algorithme.

Pour mieux appréhender notre problématique sur la stabilité de l'algorithme KMeans, nous présentons rapidement le fonctionnement de cette méthode afin de comprendre l'origine de l'aspect aléatoire qui conduit à l'instabilité.

L'algorithme est simple : nous lui indiquons un set de données et un nombre K de clusters souhaités. Prenons par exemple les points de R^2 dans le schéma en figure 11 ci-dessous représentés en noir, pour un nombre de 3 clusters. L'algorithme place ensuite K points aléatoirement

sur ce plan. C'est à ce moment que la partie aléatoire rentre en jeu. Dans notre schéma, K=3 et les points sont représentés en trois couleurs différentes : c'est l'étape 1. Ensuite, pour chaque point noir à classer, l'algorithme indique quel point coloré est le plus proche : ce point noir appartiendra ensuite à la classe de ce point coloré ; c'est l'étape 2. Une fois cette étape réalisée, on repositionne chaque point coloré afin de le placer au centre des points de sa classe, puis on réitère ces deux étapes un certain nombre de fois pour converger (les étapes 4 et 5 sont donc les mêmes que les étapes 2 et 3).

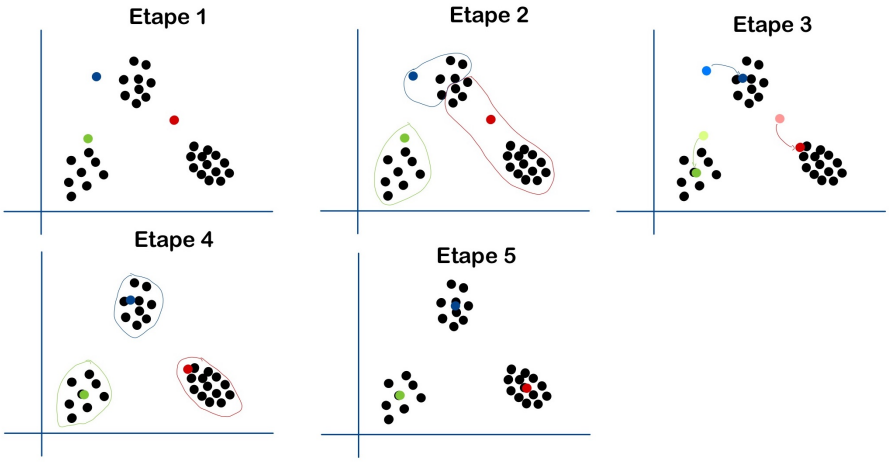


Figure 11 – Explication du fontionnement de KMeans

Un point de vocabulaire : un cluster correspond alors à un set de points noir qui sont dans la même catégorie. Le point coloré correspond alors au centroid de ce cluster.

Notre problématique est que nos données sont réparties de manière plus continue : il n'y a pas de "trou" entre les données. Si nous faisions une analogie avec des points de R^2, les données seraient plutôt réparties comme en figure 12

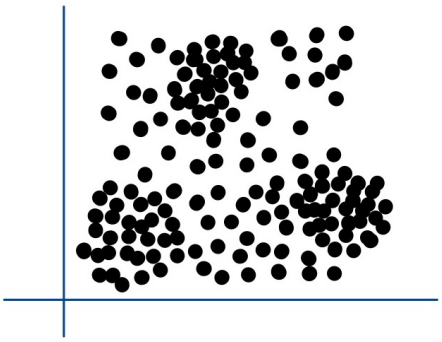


Figure 12 – Exemple de données beaucoup plus désorganisées

Sur cette figure on remarque à l'oeil trois zones un peu plus denses. Mais en fonction de l'aléatoire des premiers centroids placés, il est possible d'avoir des clusters qui restent dans des

zones moins denses, sans arriver à capter la densité d'autres zones, d'obtenir le résultat souhaité en relançant l'algorithme.

Notre enjeu est donc le suivant : comment modifier cette méthode pour créer un algorithme stable ?

2.2 Création d'un indicateur de dispersion

Une fois l'algorithme de classification de base KMeans implémenté, il s'est agi de mesurer l'instabilité des exécutions. On réalise les exécutions de l'algorithme sur un même set de données : on prend un ensemble de points obtenus comme résultats du pre-processing des données financières (processus explicité en section 1), qui va servir d'entrée à KMeans. Le premier enjeu de notre travail sur le clustering a été de quantifier la différence entre les sorties de l'algorithme. Concrètement, à chaque exécution, avec une même entrée sous forme d'un ensemble de points, nous obtenons un ensemble de catégories qui trient ces points entrés. Nous avons donc eu besoin de définir une notion de 'distance' pour comparer deux sets de clusters A et B, c'est-à-dire deux ensembles de sous-ensembles de points.

Pour chacun des points P de $\mathbf{R^n}$ donnés en entrée, on ajoute la distance euclidienne entre les deux centroids, celui du cluster auquel P appartient dans la classification A, et celui du cluster auquel P appartient dans la classification B.

Soient A, B deux sets de clusters, d la distance euclidienne, et $c_X(P)$ le centroid du cluster auquel le point P des données initiales appartient dans la classification X (A ou B).

$$dist(A, B) = \sum_{P \in \Omega} d(c_A(P), c_B(P))$$

Cette distance entre deux classifications étant établie, nous avons un moyen quantitatif de mesurer la dispersion de l'algorithme de clustering, et donc de travailler sur sa stabilité. Cette distance nouvellement définie nous a certes permis d'affirmer que KMeans introduisait un biais aléatoires, mais n'ayant pas de point de référence, il nous a été impossible de dire si ce biais était grand ou non. L'intérêt de cette mesure est qu'elle nous permettra de comparer les performances de stabilité de nos futures constructions aux performances du KMeans initial.

2.3 Intersecter des sets de clusters

Dans un objectif de stabilisation du clustering, notre première idée a été de définir une «intersection» de sets de clusters. Nous avons émis l'hypothèse que certains clusters dépendent grandement des conditions d'initialisation de KMeans, et d'autres non. On cherche à ne garder que les clusters qui apparaissent dans tous les résultats de classifications, i.e. ceux qui échapperaient au caractère aléatoire de l'algorithme. Ces clusters seraient plus fiables et plus robustes, et il s'agit donc de les isoler. Voici un exemple sur le schema ci-dessous qui vulgarise notre idée (cf figure 13). En faisant tourner deux fois l'algorithme KMeans on trouve une première fois des clusters dont les centroids sont représentés en rouge, et une seconde fois en vert. Les points

qui sont dans le même cercle bleu sont assez proches pour être considérés comme stables, et les autres points plus dispersés dépendent eux beaucoup plus des conditions initiales et ne seront donc pas gardés pour la suite. Le rayon du cercle bleu peut-être considéré comme un niveau "seuil" que l'on peut faire varier. Un premier enjeu de notre méthode est la selection de cette valeur seuil. En effet un seuil bas favorisera la précision mais diminuera le nombre de clusters restant après l'intersection.

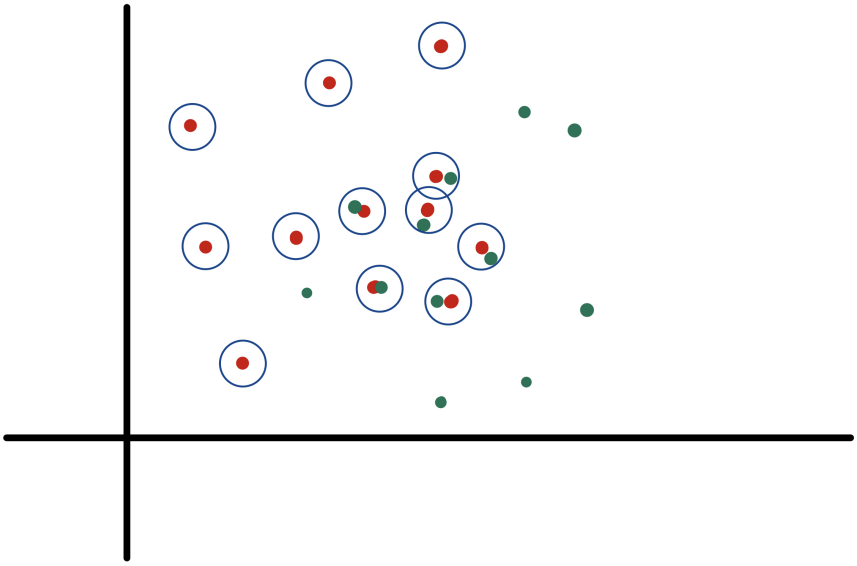


Figure 13 – Intersection de sets de clusters

Voici l'algorithme utilisé : A et B sont deux sets de clusters. *seuil* est le seuil de tolérance évoqué précédemment. Pour chaque centroid de la classification A, on regarde s'il y a un centroid de B qui est assez proche, à *seuil* près, et le cas échéant, on garde un nouveau point, moyenne pondérée des deux centroids. La pondération, matérialisée par itA, permet d'accorder un poids plus important au point de A, par exemple s'il est lui-même issu d'un processus d'intersection précédent, auquel cas il est plus fiable. Son implémentation, `filtrerClusters1` figure dans notre notebook.

2.4 La sélection de clusters

Une fois ces notions de dispersion entre les classifications prises en main et implémentées, il reste à les employer de façon pertinente, pour sélectionner les clusters les plus robustes. Pour cela, nous avons envisagé le principe suivant : comme chaque exécution de l'algorithme de clustering donnait une classification différente malgré les mêmes données et paramètres d'entrée, on peut exécuter l'algorithme de multiples fois, et ne garder que l'intersection des résultats. De cette façon, on sollicite KMeans plusieurs fois, et on se prémunit contre son instabilité : en sortie de l'algorithme d'intersection, seuls restent les clusters vraiment déterminants et stables.

A chaque intersection, on filtre un peu plus les clusters existant, et cela a pour conséquence que le nombre de clusters se réduit. L'implémentation de cette méthode peut être retrouvée sous le nom `nRaffinements2` dans notre notebook ou en Appendix.

Quand on évalue le nombre de clusters restant en fonction du nombre d'itération paramétré, on obtient le résultat figure 14. On remarque que lorsqu'on augmente le nombre de fois où on fait appel à KMeans, i.e. le nombre d'itérations, on constate que le nombre de centroids converge ! On obtient finalement, pour un seuil donné, un ensemble de points, qui ne sont pas des centroids au sens strict, car pas issus directement d'une classification. Ainsi, cela signifie qu'en itérant suffisamment l'algorithme d'intersection, on garde exactement les clusters significatifs et stables.

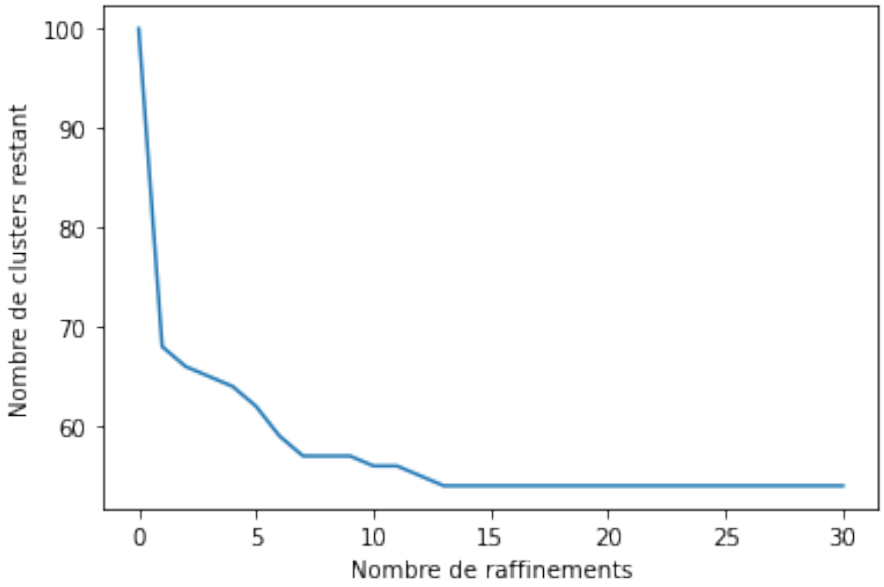


Figure 14 – Sélection des clusters les plus pertinents

Par conséquent, nous sommes maintenant munis d'une part d'une métrique pour évaluer l'efficacité des classifications, et d'autre part d'un algorithme qui permet d'obtenir des sets de clusters stables. L'enjeu est donc maintenant de voir si notre algorithme d'intersection est efficace et pertinent, en un sens que nous définirons.

2.5 L'innovation et sa paramétrisation

Très vite, il s'est révélé que notre innovation, l'idée de retenir l'intersection des sorties de notre algorithme, était conditionnée à de nombreux paramètres. Dès le départ, KMeans est un algorithme qui nécessite en paramètre le nombre de clusters pour classer les données. Ensuite, avec la méthode d'intersection que nous avons définie, nous avons souligné la nécessité de choisir un **seuil**, qui permette de déterminer si deux points sont assez proches pour être conservés ou non. Enfin, l'efficacité de notre méthode d'itération des intersections est conditionnée par le

nombre d'intersections exigé. Ainsi, quand on a à classer un set de données (vecteurs de R^n) avec notre innovation, il est subtil de choisir ces paramètres. Mais, la difficulté ici a surtout été que les sets de données en entrée étaient variables. En effet, rappelons que notre classification vient en aval du pre-processing des données. En d'autres termes, on part toujours avec les mêmes données financières, ici FRA40 sur une fraction de la période disponible 04/01/2010 - 19/07/2019, mais l'autoencodeur introduit une forte variabilité dans son exécution. Donc, les données à utiliser en entrée de clustering, issue de la sortie de l'auto-encodeur varient à chaque exécution. Par complexité de choix de ces paramètres, nous avons opté pour une paramétrisation ad hoc, propre à chaque exécution, où nous fixons à la main les paramètres. Dans cette sous-section, nous présentons donc les sensibilités des différents paramètres, la méthode pour déterminer des valeurs pertinentes. Nous suggérerons ensuite une amélioration sur les méthodes de choix de ces paramètres.

2.5.1 • Le nombre de clusters

Dans le choix du nombre de clusters, la difficulté est double. Premièrement, les appels à KMeans exigent qu'on donne un nombre de clusters pour trier. Ensuite, notre méthode de raffinements par intersection fait diminuer le nombre de clusters final. Par conséquent, si on souhaite un nombre de clusters précis en sortie de notre algorithme, il faut aussi surévaluer le nombre de clusters exigé lors de nos appels à KMeans dans l'intersection. Pour estimer le nombre de clusters pertinent pour notre set de données, nous avons utilisé une méthode connue dans la littérature, la méthode du coude [8]. Il s'agit de mesurer la variance de notre classification, déjà évoquée en 2.2, en fonction du nombre de clusters paramétré. La courbe de sortie ressemble à une hyperbole, et il s'agit de trouver son coude : au niveau du coude, l'ajout d'une classe (i.e. d'un cluster) supplémentaire n'est plus pertinent. Autrement dit, rajouter des classes ne permet pas d'obtenir un gain siginificatif dans la classification. Ainsi, on détermine graphiquement le nombre de clusters idéal sur la figure 15.

Ce nombre de cluster, appelons le n_f, est celui à atteindre idéalement en sortie d'algorithme d'intersections. Ici, dans l'exemple de FRA40 sur la période spécifiée, on relève un nombre approximatif de 40. Pour notre méthode `nRaffinements`, nous devons donc exécuter KMeans avec un nombre plus important de clusters, qu'il faut donc déterminer et qu'on nommera n_i.

2.5.2 • Le seuil de tolérance des intersections

La détermination du nombre de clusters à fournir à KMeans n_i se fait en parallèle de la détermination du seuil s à fixer pour nos intersections. Rappelons que ce seuil permet de déterminer si un point de la classification (centroid) est assez proche d'un point de l'autre classification (à seuil près), et à conserver une moyenne des deux points le cas échéant, et à éliminer le point sinon. Diminuer s permet d'être plus sélectif sur les clusters à garder, mais va avoir pour effet de diminuer le ratio n_f/n_i. Comme on cherche à cibler n_f, diminuer s implique d'augmenter n_i et inversement. Pour déterminer ces deux valeurs, nous avons procédé par essais successifs, en utilisant deux graphiques. Le premier, déjà présenté en figure 14 permet de

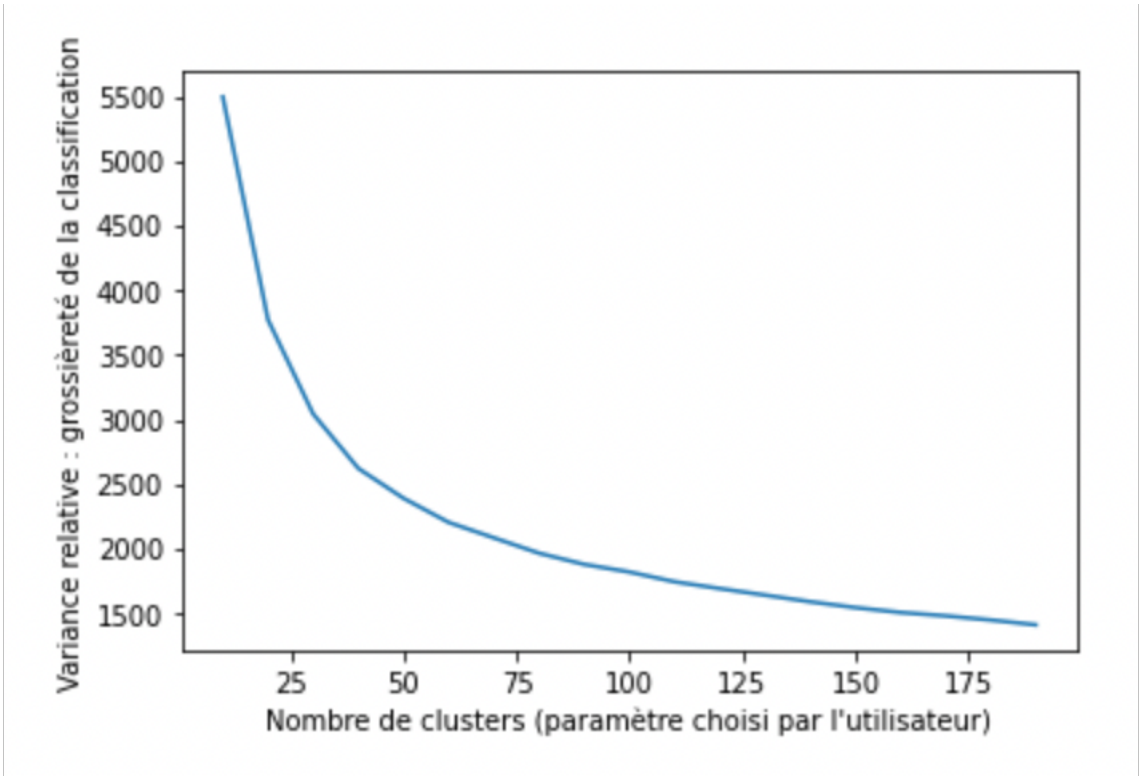


Figure 15 – Détermination du nombre de clusters

déterminer le nombre de convergence n_f, à s et $n_i donnés$. Parallèlement, on s'appuie sur la figure 16 pour trouver un seuil s qui soit cohérent avec n_i. En reprenant l'exemple de FRA40, sur la période spécifiée, on observe avec quelques essais que $s = 0.3$ et $n_i = 80$ correspondent. Ici, cette méthode peut être améliorée. Déterminer, à l'aide de graphes, au cas par cas, les paramètres à déterminer est peu optimal. Ainsi, une suggestion serait d'automatiser la recherche des meilleurs paramètres, par exemple avec Optuna, suggestion sur laquelle on reviendra en 2.8. Néanmoins, gardons en mémoire que nos algorithmes mettent quelques minutes à tourner sur nos machines personnelles. Même sur des outils professionnels, on imagine que l'optimisation, faisant de multiples appels à notre algorithme de classification, serait extrêmement coûteux en temps.

2.6 Le nombre de raffinements

Nous avons déjà constaté avec la figure 14 que le nombre de clusters convergeait, pour un nombre assez important d'itérations. Ainsi, dans l'idéal, le nombre d'itérations de l'étape d'intersection est infini. Cependant, à chaque itération, on appelle KMeans, ce qui est coûteux en temps et en ressources pour la machine. Donc, le nombre d'itérations doit rester raisonnable. En accord avec la figure 6, un nombre d'itérations $n_{itérations} = 25$ semble pertinent, et raisonnable en coût.

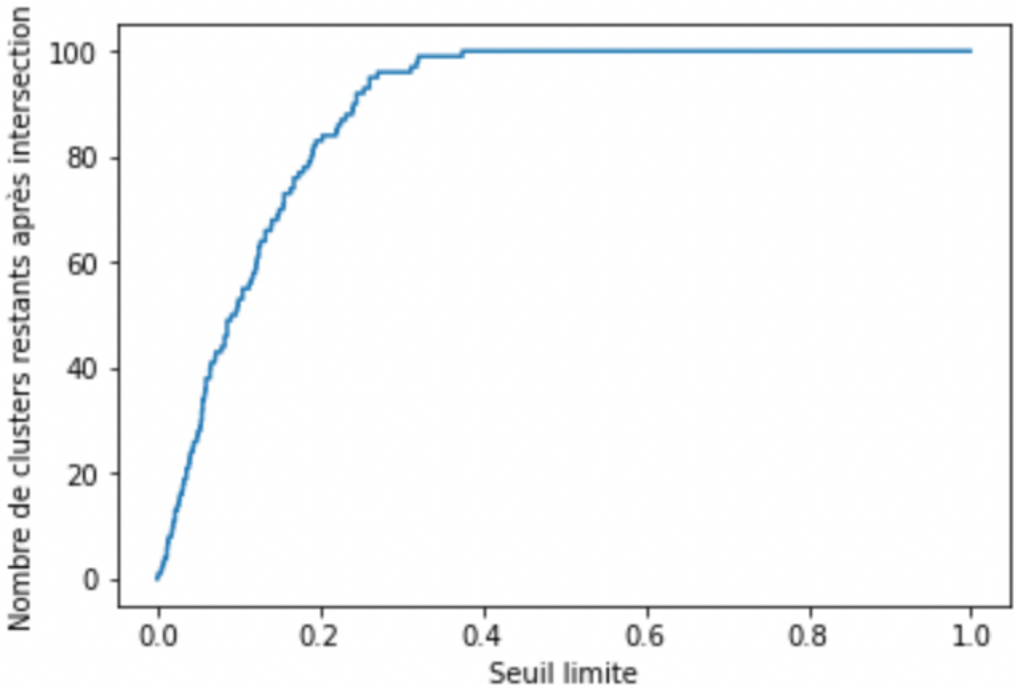


Figure 16 – Nombre de clusters après l'intersection de deux exécutions, en fonction du seuil de tolérance choisié

2.7 Les résultats obtenus

En choisissant les paramètres de notre algorithme avec les méthodes décrites en 2.5, notre algorithme fournit des réultats significatifs, quantifiables. Pour évaluer le gain apporté par notre innovation, nous avons utilisé la méthode suivante :

- Sur un même set de donnée initial (en sortie de l'auto-encodeur), on exécute deux fois séparément l'algorithme de KMeans classique. On mesure ensuite la dispersion, au sens de la variance définie en 2.2, entre les deux classifications obtenues.
- Toujours sur les mêmes données, on exécute deux fois séparément notre algorithme de raffinements, avec les paramètres choisis dans 2.5. De même, on mesure ensuite la variance entre les deux classifications.
- Ensuite, on compare les deux variances obtenues par chaque méthode, pour évaluer le gain de notre innovation.
- Finalement, on répète ces étapes plusieurs fois, pour obtenir un gain moyen. (5 fois dans le tableau)

Cette méthode permet donc de comparer l'instabilité, i.e. le caractère aléatoire, de l'algorithme classique KMeans, et de notre méthode de raffinements. Nous avons donc implémenté cette méthode sur Python, et le programme génère dans un fichier `.txt`, qu'on interprète de la façon suivante :

Essai	Variance avec méthode d'intersection	Variance avec méthode KMeans classique	Gain
1	1409	1812	22.2%
2	1688	1826	7.5%
3	1336	1474	9.4%
4	1410	1731	18.5%
5	1625	1788	9.1%
Moyennes	1494	1726	13%

Par conséquent, dans cette exemple, nous voyons que notre innovation améliore à chaque comparaison des 2 algorithmes (5 comparaisons réalisées ici) la stabilité de la classification. Ce gain en stabilité est d'en moyenne 13%, et peut même monter à 22%.

2.8 Conclusion et perspectives

Ainsi, partant de l'idée de réaliser l'intersection des classifications obtenues, nous avons défini précisément le sens d'une intersection de sets de clusters, et l'avons utilisé pour raffiner un set existant. Cette méthode est soumise à un choix sensible des paramètres, mais montre des résultats pertinents si ces paramètres sont choisis. En effet, la stabilité de classification est améliorée de 13%, traduisant une meilleure répétabilité des résultats.

En théorie, cette répétabilité va permettre de trouver des clusters plus significatifs, et donc d'associer un nouveau point à son cluster de façon plus pertinente. En d'autres termes, lorsqu'on aura à classifier une portion de courbe financière parmi les catégories existantes (les clusters), le choix de la catégorie sera plus sûr, et donc la prédiction sur l'évolution du marché, elle aussi plus sûre. In fine, cela permettra de passer des ordres de vente ou d'achat avec plus de confiance, et donc de diminuer le risque de la position, ou de prendre de nouvelles positions, auparavant incertaines, et finalement d'augmenter la valeur de notre portefeuille plus rapidement, et de façon plus fiable. Pour voir si cette innovation permet en effet de gagner de l'argent ou non, le backtesting réalisé en 3 se révèle essentiel.

3 PRÉDICTION ET BACKTESTING

Jusqu'à présent, nous disposons d'un algorithme de classification qui est plus stable qu'un modèle KMeans normal, ainsi que d'un auto-encodeur qui nous permet de réduire la dimension des données d'entrée. Nous allons maintenant essayer de voir à quel point ces modèles sont pertinents quant à la prédiction des tendances de marché afin de construire une stratégie de trading robuste. Pour ce faire, on développera dans un premier temps deux filtres sur les clusters obtenus pour ne garder que les patterns qui nous intéressent. Puis, on procédera à un backtesting sur le marché en tenant en compte plusieurs facteurs de la finance de marché de façon qu'on évaluera correctement le caractère prédictif de notre modèle.

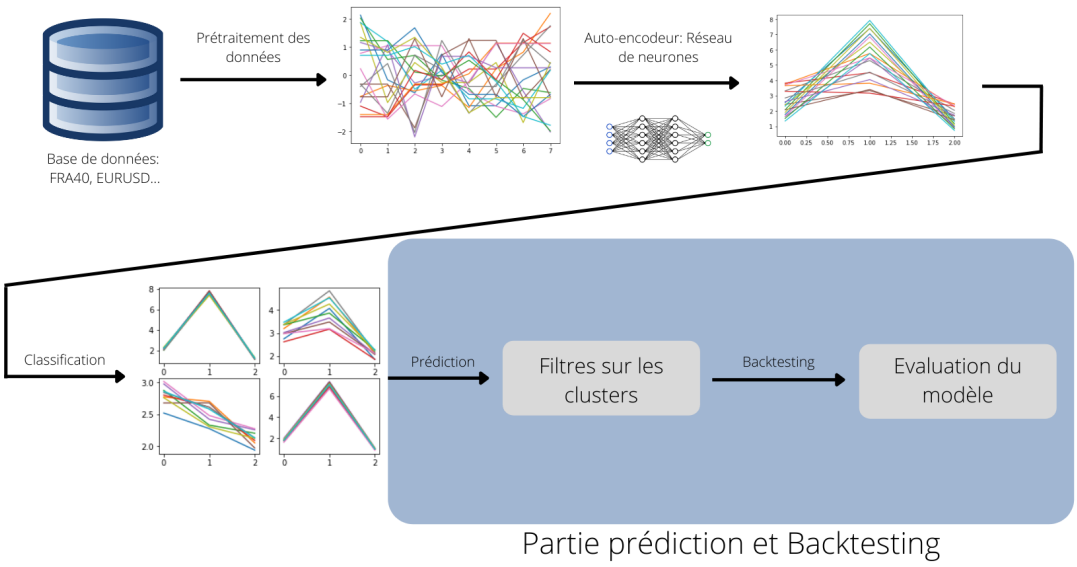


Figure 17 – Sommaire du modèle

3.1 Prédiction

3.1.1 • Principe

Avant d'utiliser le modèle de classification qu'on a développé auparavant, il est indispensable de procéder à un filtrage des clusters obtenus pour ne garder que les clusters qui sont prédictifs

et pertinents. Le filtrage conciste à classer tous les clusters pour dire s'ils ont des chances de prédire une hausse, une baisse, ou s'ils sont trop incertains pour investir dans le futur. En effet, l'approche des raffinements entre les centroïdes des clusters utilisés pour le clustering prend en compte un aspect topologique loin de toute interprétation financière. Ainsi, pour que notre modèle soit plus pertinent - d'un point de vue financier d'une part et d'un point de vue mathématiques d'autre part - nous avons envisagé d'appliquer aux clusters un filtre qui met l'accent sur cet aspect financier.

Une première approche consiste à considérer le nombre d'éléments de chaque cluster comme critère de filtrage. On ne gardera ainsi que les clusters dont le cardinal dépasse une certaine valeur minimale que nous allons fixer selon le jeu de données étudiées. En effet, nous considérons les clusters qui ont un cardinal élevé plus fiables que les autres car le pattern qui caractérise le cluster en question a été observé plus de fois.

Cette approche reste efficace mais elle ne reflète pas toujours l'aspect financier. On a alors cherché une métrique qui décrit l'aspect financier d'un cluster sur laquelle on va baser le deuxième filtre. Après concertation avec Lusis, et dans la continuité du travail effectué l'année dernière, nous utilisons le PIP comme critère d'évaluation. Notre utilisation du PIP permettra lors de l'entraînement des données d'estimer la tendance après une séquence financière.

Pour expliquer ce qu'est le PIP, commençons par expliquer le fonctionnement de nos investissements : nous investissons pour une durée que nous fixons à l'issue de la phase d'entraînement des données $T_{tracking}$. Cela signifie qu'au moment ou l'on choisit d'investir, nous entrons dans le trade à la date actuelle pour n'en sortir qu'après $T_{tracking}$ unités de temps. Ce $T_{tracking}$ est essentiel car le PIP donne une indication sur la tendance à l'issue d'une séquence sur une durée égale à $T_{tracking}$. La figure 18 ci-dessous représente graphiquement ce qu'est le PIP et l'utilisation du $T_{tracking}$ pour une séquence. Le PIP d'un cluster sera ensuite la moyenne des PIP des séquences qui composent le cluster. On rappelle que le PIP est calculé sur les données d'entraînement et que l'on connait donc la valeur des cours représentés en pointillés ici. Un lecteur avisé trouvera l'algorithme qui calcule le PIP en Appendix.

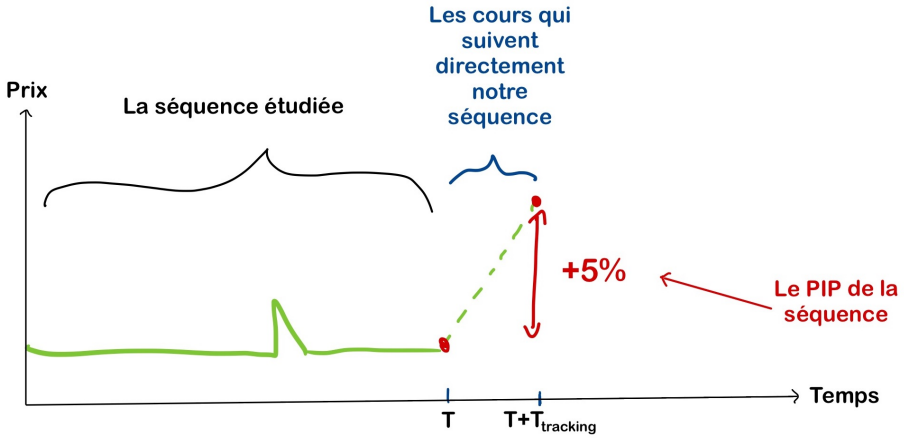


Figure 18 – Explication du PIP sur une séquence

Il faut donc décider du temps optimal $T_{tracking}$ que nous utiliserons. Nous avons envisagé de calculer les PIPs des clusters pour différents intervalles de temps et choisir ensuite ce temps optimal.

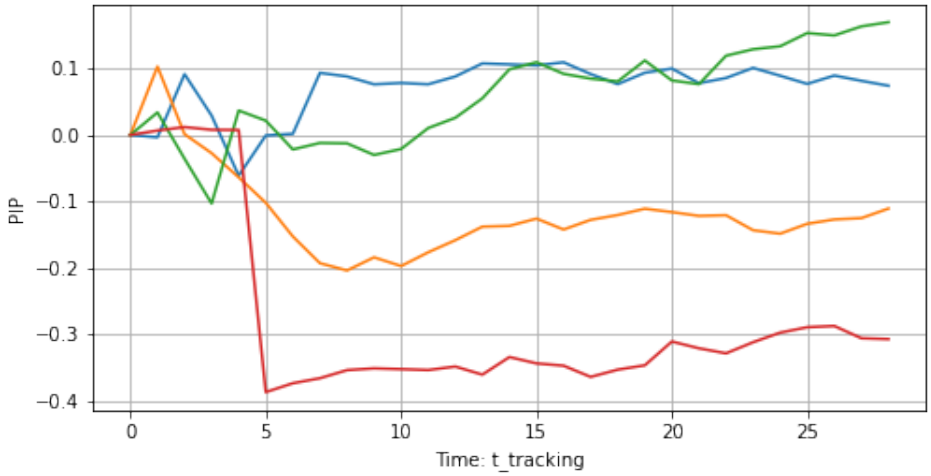


Figure 19 – Représentations de la variation de PIP pour quatre sets de cluster différents

La figure 19 montre le PIP de quatre clusters du 'FRA40' différents en fonction de la valeurs de $T_{tracking}$ qui varie entre 0 et 2h30min. Une valeur de PIP négative reflète une perte à anticiper, ce qui est le cas des deux clusters représentés en rouge et en orange, alors qu'une valeur de PIP positive permet d'anticiper un profit prochainement, situation qu'on observe avec les deux clusters en couleur bleue et en couleur verte. De plus, ces pertes et ces profits n'ont pas la même ampleur étant donnée les différentes valeurs de PIP associées ; pour un $T_{tracking} = 25$ le cluster rouge prédit une perte plus importante que le cluster orange.

Finalement, pour le choix de temps $T_{tracking}$, nous choisissons le temps qui gardera le plus de clusters après application des deux filtres précédemment explicités.

3.2 Backtesting sur le marché

Le backtesting constitue une composante indispensable dans l'étude des modèles quantitatifs en finance. C'est le test de notre stratégie. Il permet en effet de savoir à quel point le modèle est pertinent pour décider par la suite si on peut l'utiliser concrètement ou plutôt si on doit encore l'améliorer et corriger ses défauts constatés lors du backtesting. On distingue principalement deux usages du backtesting. Le premier concerne les modèles d'évaluation des risques liés à la gestion des portefeuilles [9], où on fait recours à des grandeurs quantitatives telle que la VaR – Value at Risk. Le deuxième usage du backtesting, qui est notre cas, concerne les stratégies du trading [10][17] dont l'objectif est de mettre la stratégie en épreuve avec des données récentes.

La logique derrière est que si la stratégie a fonctionné auparavant, elle a une bonne mais pas certaine chance de fonctionner à nouveau dans le futur et inversement si le concept n'a pas bien fonctionné dans le passé, il ne fonctionnera probablement pas bien à l'avenir. Le backtesting dans le cas du trading repose aussi sur différents indicateurs qui décrivent la performance de la stratégie. Dans ce qui suit nous allons utiliser ces cinq indicateurs :

- **Sharp Ratio :** un outil pour observer dans quelle mesure il est plus intéressant de mettre son argent sur cette stratégie au lieu de la prêter aux banques. Plus le Sharpe est positif, plus la stratégie est pertinente.
- **Max Drawdown :** une façon de quantifier les risques liés à cette stratégie en évaluant la perte attendue la plus forte durant toute la période du backtesting.
- **Winning rate :** c'est le taux de réussite, il représente le nombre de trade qui ont entrainé un gain et pas une perte.
- **Benchmark :** représente le revenu réel du marché entre le début et la fin du backtesting.
- **Return** évalue le revenu touché par cette stratégie après la fin du backtesting.

De plus, pour que notre backtesting soit plus réaliste, nous allons prendre en compte le spread ainsi que le taux d'intérêt nominal. Pour des raisons de simplification, nous utilisons les moyennes de ces valeurs durant l'année 2017.

Dans la suite nous allons présenter le backtesting du modèle sur l'indice du '$FRA40'$ pour comparer les résultats de notre modèle avec le modèle réalisé l'année dernière. Dans l'Appendix les résultats de notre modèle sur d'autres indices comme sur 'EUR/USD' sont présentés, ainsi que le résultat du backtesting sur un modèle de classification établie sur deux indices.

3.2.1 • Backtesting sur l'indice FRA40

• Description du jeu de données

Le jeu de données considéré ici est constitué de la variation du prix d'ouverture de l'indice '$FRA40'$ entre 2012 et 2017. On a pris la période 2012-2016 comme période d'apprentissage du modèle et donc le backtesting va être effectué sur l'année 2017. Il est à noter que la dimension des données est sous la forme (Taille d'échantillon, 8) ; il s'agit d'un ensemble de donnée de 8 points chacune.

FRA40	Date de début	Date de fin	Paramètre	Période	Dimension des données
Train set	2012-01-04	2016-01-04	open	40min	(20551, 8)
Test set	2016-01-04	2017-01-04	open	40min	(5421, 8)

• Optimisation des paramètres

Avant de faire tourner l'algorithme du backtesting, nous devons optimiser les paramètres de notre modèle. Pour le clustering : le nombre de clusters et le seuil souhaité pour les raffinements

(voir la partie clustering) - puis pour la filtration les paramètres qui concernent la filtration des clusters.

- **Prédiction**

Pour la prédiction nous devons optimiser trois paramètres : le temps de suivi $T_{tracking}$, la valeur minimale du PIP MIN_PIP ainsi que le nombre d'éléments minimal contenu dans un cluster. La fonctionnelle de l'optimisation que nous allons considérer est le nombre de cluster qu'on gardera après le filtrage.

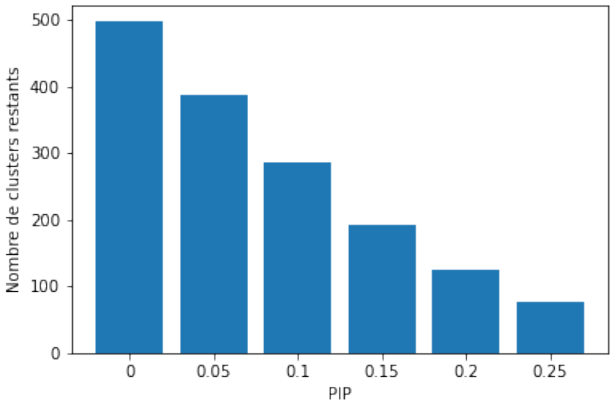


Figure 20 – Nombre de clusters conservés en fonction de la valeur MIN_PIP choisie

On a constaté qu'il est difficile d'optimiser ces trois paramètres à la fois, de ce fait on a décidé de fixer la valeur de $T_{tracking} = 20$ et d'optimiser les autres paramètres selon le nombre de clusters conservés à la fin. Avec cette valeur de $T_{tracking}$, on décide de ne faire des prédictions sur le marché que pour une période de 20 unités de temps, ce qui donne pour notre cas à peu près 2h. En ce qui concerne le paramètre MIN_PIP, nous utiliserons la valeur 0.05 pour filtrer à peu près un cinquième des clusters.

Pour le dernier paramètre, on va prendre la valeur $MIN_ELEM = 45$ qui dépasse à peine le nombre d'éléments si on suppose une répartition uniforme entre les clusters. Finalement on conservera plus de 120 clusters.

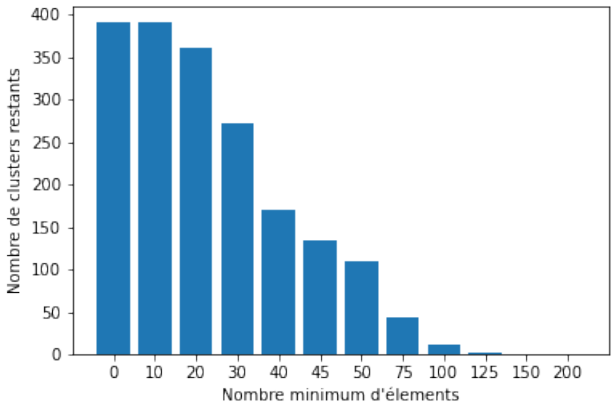
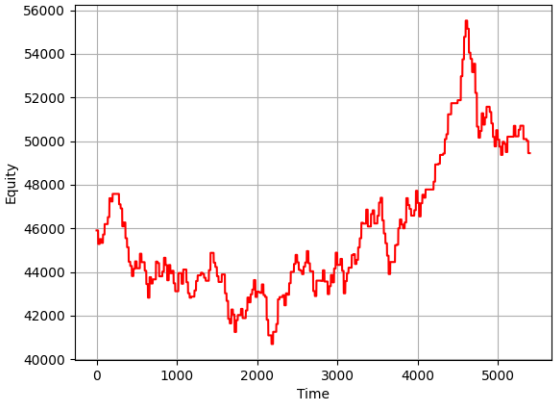


Figure 21 – Nombre de clusters conservés en fonction de la valeur MIN_PIP choisie

Ainsi, nous avons constaté que les deux valeurs optimales de ces paramètres sont $(T_{tracking}, MIN_PIP) = (20, 0.05)$, ce sont les deux valeurs que nous considèrerons pour la suite. Ainsi, après les deux filtres nous conserverons 321 clusters.

• Résultats

En faisant tourner l'algorithme du backtesting sur l'année 2017, on obtient les résultats suivants :



Nombre de trade initial	10
Number of predictive clusters	127/500
Winning rate	0.52%
Trades	203
Sharpe rate	2.4
Max Drawdown rate	2
BenchMark rate	7.6%
Return	11.8%

Figure 22 – Résultats du backtesting sur le FRA40 durant l'année 2017

La courbe de la variation de l'equity montre l'évolution de 10 trades initiaux que nous avons mis au début sur le FRA40. Après la période d'une année on ressort avec un gain de 11.8%, relativement plus supérieur que le Benchmark. D'autant plus, le sharpe ratio est à 2.4, ce qui signifie qu'investir sur cette stratégie a plus de gain que de prêter de l'argent à une banque à un taux de 1%.

Quant au risque lié à la stratégie, le Max Drawdown est évalué à 2%, qui est la perte maximale à laquelle nous faisons face durant cette période, mais la courbe de l'equity démontre qu'il s'agit uniquement d'une décroissance limitée dans le temps avant que les cours ne repartent à la hausse. Ainsi ce taux de perte reste acceptable pour une stratégie de trading sur cette période de temps envisagée.

Mais ce n'est pas tout, la chose la plus importante qu'on a gagnée avec ce modèle est la stabilité. En effet, le modèle crée l'année dernière n'était pas stable dans la mesure où il ne renvoyait pas les mêmes résultats si on le faisait tourner plusieurs fois (voir Appendix). Cette instabilité était flagrante ; nous pouvions avoir deux allures de la variations de l'equity totalement différentes entre les deux exécutions. Nous pouvons donc conclure que l'approche des raffinements entre les centres de clusters est efficace. En effet, notre modèle est plus stable même avec un input de taille plus grande. En Appendix plusieurs courbes de l'equity obtenues en faisant tourner le programme plusieurs fois démontrent bien cette stabilité que nous avons obtenue.

Néanmois, nous avons constaté que la période d'apprentissage de notre modèle est plus longue. En effet, pour avoir un classificateur stable, l'approche par raffinements est très longue car elle utilise plusieurs fois l'algorithme KMeans. Nous avons quantifié les durées d'exécution de tout le modèle jusqu'au backtesting en fonction du nombre d'itérations effectuées pour les raffinements :

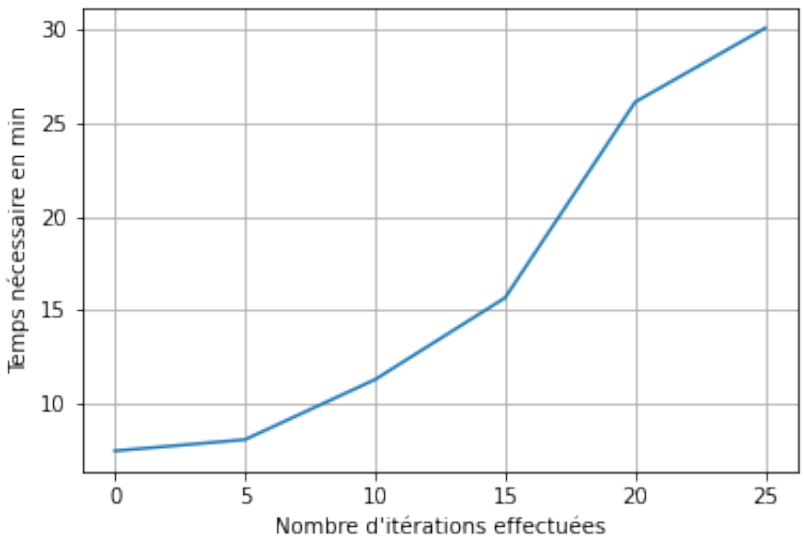


Figure 23 – Complexité temporelle du modèle en fonction du nombre d'itérations

On observe alors qu'avec 25 itérations, nous avons besoin d'environ 30 minutes pour faire tourner le modèle par rapport à moins de 5 minutes si on utilise un KMeans simple. Toutefois çe n'est pas un problème en soi, car une fois le modèle entrainé la phase de la mise en pratique de la stratégie de trading ne requiert pas beaucoup de temps ; cela se fait en quelques minutes

si on considère une longue période de temps.

Il ne faut pas non plus oublier l'effet indispensable des autoencodeurs qui nous a permis la réduction de la dimension des données d'entrée. Cela réduit remarquablement les temps d'exécution du modèle.

4 CONCLUSION

Le cahier des charges de nos tuteurs de LUSIS a été rempli. Nous avons réussi à créer une stratégie gagnante qui soit stable. Nous avons pour cela travaillé sur la stabilité de la classification des données : d'abord grâce aux auto-encodeurs, puis en adaptant l'algorithme KMeans.

L'étape du *preprocessing* a permis de compresser très significativement nos vecteurs d'entrée, accélerer de façon décisive les temps d'éxécution et nous a permis de balayer un panel de différentes techniques : les objectifs ont été atteints. Nous avons défini et identifié au cours du projet la problématique de l'instabilité des auto-encodeurs et nous avons mis en place des outils pour mesurer cette instabilité ; nonobstant, cette notion peut encore être approfondie avec l'utilisation d'auto-encodeurs plus pointus (variationnels) : il y a encore des axes de recherche pour une extension de notre travail par les groupes suivants.

Concernant le clustering, la stabilité de classification a été améliorée de plus de 10% par rapport à la méthode KMeans classique, traduisant une meilleure répétabilité des résultats, et répondant donc à l'objectif initial d'augmentation de la stabilité des exécutions de la phase de clustering. Notre méthode utilise une notion d'intersection de clusters obtenues par plusieurs exécutions de l'algorithme connu KMeans. Nous avons donc défini précisément le sens d'une intersection de sets de clusters, et l'avons utilisée en l'itérant pour raffiner un set existant. Cette méthode s'est révélée soumise à un choix sensible des paramètres, mais a montré des résultats très intéressants, en choisissant les bons hyperparamètres.

Notre projet s'inscrivant dans le cadre d'une continuité entre les années, nous envisageons plusieurs pistes d'amélioration. Premièrement, nous avons vu dans 2.5 que notre modèle était sensible aux hyperparamètres, que nous avons fixés manuellement. Il pourrait être pertinent d'optimiser ce choix automatiquement, par exemple en utilisant des packages Python tels qu'*Optuna*[5][6]. Ensuite, nous avions imaginé intersecter des données de marchés différents. En effet, intersecter des clusters issus de EUR-USD et de FRA40 pourrait être intéressant. On imagine, en intersectant, ne garder que des clusters qui seraient significatifs sur chaque marché, et universels en ce sens. Cette piste pourrait encore améliorer la fiabilité des prédictions, et ces clusters "universels" pourraient ensuite peut-être être utilisés pour des prédictions sur un nouveau marché, sur lequel l'apprentissage n'a pas été réalisé.

Les résultats du backtesting sont sans appel concernant l'efficacité de nos méthodes. Nous réussissons à battre le marché dans la durée, en répétant l'expérience un grand nombre de fois. Le caractère prédictif des clusters a plus de sens dans la mesure où les clusters seront quasiment les mêmes à chaque exécution de l'algorithme. Pour classer les clusters en fonction de leur predictabilité, nous avons donc utilisé le PIP pour caractériser la variation des cours suite à des patterns du cluster en question, ainsi que du nombre de séquences présentes dans le cluster. Ces critères jugent l'aspect financier des clusters.

Cependant les cours de l'equity que nous avons obtenus incluent également de longues phases de baisse. Dans la continuité du projet, il pourrait être intéressant de coupler notre travail sur la classification des time series avec un autre indicateur. Par exemple : acheter une action si notre algorithme indique que nous sommes sur une time series qui prédit une hausse, et que nous venons de rencontrer un GAP haussier. Coupler des méthodes diminue le nombre de trade, mais permet d'augmenter en fiabilité.

APPENDIX

Lien de nos codes implémentés

Veuillez trouver l'implémentation Python de notre travail au lien suivant : `https://www.dropbox.com/s/mo4xizotcxk03dc/ECO04_Code_Source.zip?dl=0`

Organisation du travail et calendrier

- ### Organisation du travail

Le projet consistant à analyser les «time series» à l'aide du machine learning, il a d'abord été nécessaire de comprendre ce qu'est le machine learning. Il s'agissait ainsi de dégager au fur et à mesure les différentes problématiques qu'il fallait résoudre pour atteindre notre objectif. Nous avons alors réparti l'essentiel des tâches en trois groupes de travail.

Le premier groupe, composé de Meilame Tayebjee, Florian Labaye et Abdelhamid Jenzri, s'est consacré à la préparation des données afin qu'elles puissent être exploitées par la suite. Des réunions hebdomadaires avec notre tuteur nous ont permis d'avancer sur le sujet complexe des auto-encodeurs. Chaque réunion était suivie d'une liste de tâches à réaliser pour la semaine suivante, que nous nous répartissions au sein du groupe auto-encodeurs.

Le second groupe, constitué de Louis Lorrach et Sacha Braun, s'est occupé de l'exploitation des données à travers des méthodes de clustering. Différentes méthodes ont été implémentées, et différentes idées ont été testées pour améliorer la stabilité de l'algorithme. L'idée d'intersection s'est particulièrement dégagée, et a constitué le coeur du travail de ce groupe.

Enfin, Mohamed Lakhnichi, en collaboration étroite avec le deuxième groupe, a réalisé la mise en place des tests-rétroactifs pour tester la pertinence du modèle de classification produit. C'est ce qu'on appelle le backtesting. La principale difficulté était de faire attention aux flux des données et le changement des dimensions que cette partie engendre. Il ne faut pas oublier le problème d'ajouter des critères pour rendre le modèle conforme aux concepts économiques du trading. Les entretiens hebdomadaires avec notre tuteur étaient très importants, et ils ont permis de surmonter les difficultés rencontrées.

- ### Calendrier des tâches

Septembre - Novembre : prise de recul et définition des objectifs

— Étude du PSC réalisé l'année dernière, de ses limites et des points qui pourraient être améliorés.
— Formation technique en suivant les MOOCs "Deep Learning" sur Coursera, que Lusis nous a recommandé. Ces cours nous ont permis d'acquérir des bases solides dans le domaine du Machine Learning.
— Compréhension des processus de formation des prix pour prendre du recul sur le sujet et apporter un regard critique sur les futurs résultats.

Décembre - Janvier : début du travail par groupe

— Groupe 1 : implémentation d'un premier auto-encodeur sur Python avec l'aide de Lusis. Le problème majeur que nous avons rencontré a été d'améliorer la stabilité de l'auto-encodeur.
— Groupe 2 : prise en main de KMeans et son implémentation Python. Développement des premiers algorithmes de clustering. Rassemblement d'idées à mettre en oeuvre pour améliorer la stabilité et premiers tests.
— Groupe 3 : mise en place des filtres pertinents pour choisir les clusters les plus prédictifs pour le modèle de backtesting.

Février - Avril : approfondissements et mise en relation des travaux

— Groupe 1 : résolution de la question des hyperparamètres à utiliser pour l'auto-encodeur grâce à l'utilisation de la bibliothèque Optuna ; implémentation de modèles déterministes afin de faire face au problème de stabilité de l'auto-encodeur.
— Groupe 2 : détermination de l'innovation (l'intersection) à garder après essais, puis choix des hyperparamètres. Evaluation des bénéfices apportés par l'innovation.
— Groupe 3 : mise en production du modèle de backtesting principalement sur l'indice FRA40, et comparaison de nos résultats avec les résultats de l'année dernière.

Implémentation de la méthode par raffinements

```
def nRaffinements2(nit,nbClusters,seuil,Liste):
    KMeans = clusters(nbClusters)
    y = KMeans.fit_predict(Liste)
    A = KMeans.cluster_centers_
    long = [len(A)]
    for i in range(nit):
        KMeans2 = clusters(nbClusters)
        y2 = KMeans2.fit_predict(Liste)
        B = KMeans2.cluster_centers_
        A = filtrerClusters1(A,i,B,seuil)
        long.append(len(A))
return (A,long)
```

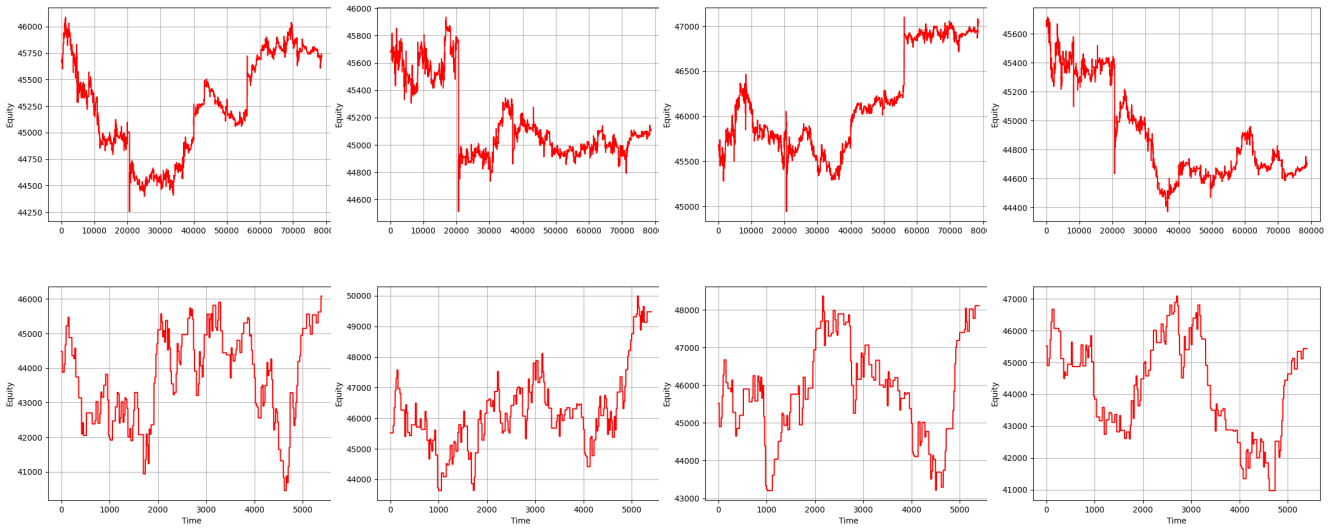
Algorithme de calcul du PIP

Algorithm 1: PIP_cluster($T_{tracking}, history_prices, nb_cluster$)

$PIP \leftarrow [0] \times T_{tracking}$;
$count \leftarrow 0$;
for *All patterns p belonging to the cluster nb_cluster* **do**
  $prices \leftarrow history_prices[pattern_date, pattern_date + T_{tracking}]$;
  Add prices to PIP element by element;
  $count \leftarrow count + 1$;
end
return $PIP/count$;

Stabilité du backtesting

Différentes executions de notre modèle de backtesting en comparaison avec celui de l'année dernière ; les plots en haut correspondent aux résultats du modèles de l'année dernière, alors que ceux en bas montrent les résultats obtenus par notre modèle :



Backtesting sur l'indice EUR/USD

• Description du jeu de données

On s'intéresse maintenant à la paire Forex EURUSD, on va changer les intervalles de temps par rapport à ce qu'on a fait avec le FRA40. Mais quant à la dimension de l'entrée on va la conserver :

EURUSD	Date de début	Date de fin	Paramètre	Période	Dimension des données
Train set	2012-01-06	2016-01-04	open	40min	(75357, 8)
Test set	2016-01-06	2017-01-06	open	40min	(5421, 8)

• Optimisation des paramètres

On va suivre les mêmes démarches pour optimiser les paramètres nécessaires, ainsi on ne va pas entrer beaucoup dans les détails, on se contentra de montrer les valeurs des paramètres que nous allons prendre avec le support graphique correspondant

- **Modèle de classification**

Pour le paramètre k, avec la méthode du coude on a trouvé une valeur pertinente de 500 :

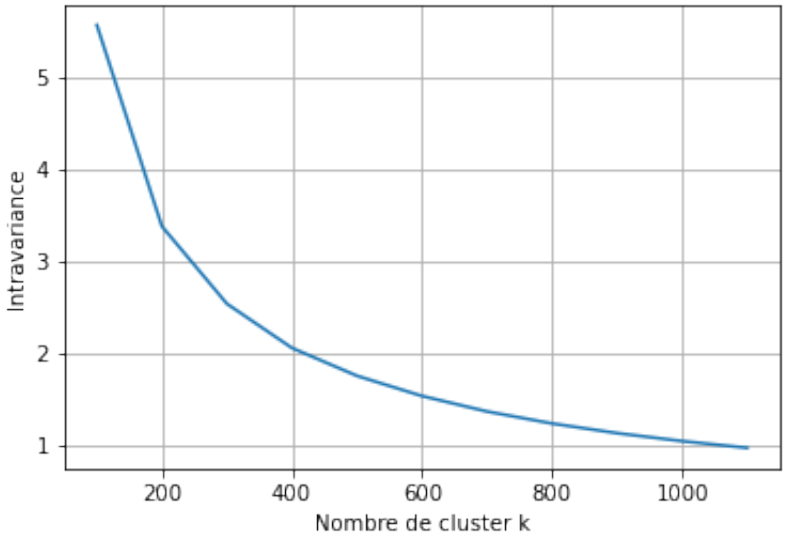


Figure 24 – Intravariance des clusters en fonction du nombre k

- **Prédiction**

En ce qui concerne les paramètres de la prédiction, on fixera $T_{tracking} = 20$, et pour les autres paramètres nous allons prendre : $(MIN_PIP, MIN_ELEM) = (0.15, 20)$

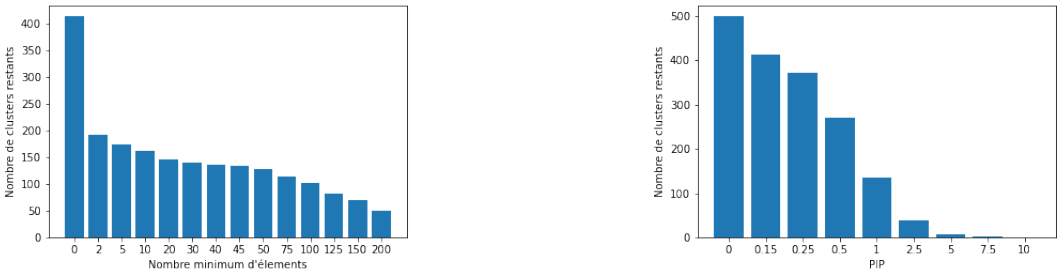
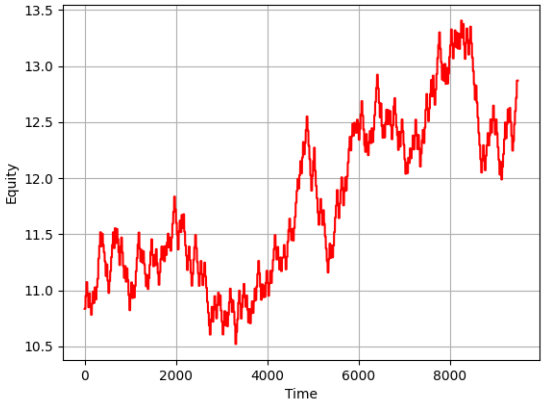


Figure 25 – Le nombre de clusters restant en fonction de PIP et de MIN_ELEM

- ## Résultats

On a exécuté l'algorithme de backtesting sur la période de l'année 2021, et nous avons obtenus les résultats suivants en figure 26.

Nous nous félicitons des performances positives de notre algorithme malgré le contexte économique de l'année 2021 : une année 2021 très volatile et imprévisible. Malgré ce caractère imprévisible dans un contexte de crise intense, notre modèle a fonctionné. Cependant, même si notre modèle reste gagnant, le nombre de données a été multiplié par 3 ce qui se traduit par une légère baisse de l'efficacité des techniques utilisées pour travailler la stabilité de notre algorithme.



Nombre de trade initial	10 000
Number of predictive clusters	498/500
Winning rate	0.53%
Trades	473
Sharpe rate	2.6
Max Drawdown rate	2%
BenchMark rate	-1.6%
Return	18.8%

Figure 26 – Résultats du backtesting sur l'EURUSD durant l'année 2021

Backtesting sur le FRA40 et EURUSD à la fois

• Description du jeu de données

Dans cette partie, nous allons prendre les données et de l'historique du FRA40 et de l'historique de la paire EURUSD. De ce fait, on va considérer la période 2012-2016 pour l'apprentissage du modèle pour faire par la suite le backtesting sur l'année 2017.

FRA40 et EURUSD	Date de début	Date de fin	Paramètre	Période	Dimension des données
Train set	2012-01-06	2020-01-04	open	40min	(41102, 8)
Test set	2020-01-06	2021-01-06	open	40min	(5421, 8)

Le classificateur va être le résultat de l'intersection entre plusieurs centroides de classificateurs obtenus après apprentissage soit sur le FRA40 soit sur l'EURUSD. En d'autres mots, à chaque itération on prendra deux modèle KMeans ; le premier on le fait tourner sur le FRA40 alors que le deuxième prend la paire EURUSD. On fera par la suite l'intersection entre ces deux modèles. A chaque itération on répétera le même processus jusqu'à obtention du dernier modèle de classification au bout de la 25ème itération.

• Optimisation des paramètres

Pour ce modèle, nous allons prendre les paramètres optimisant chaque indice que nous avons trouvés dans les parties précédentes. Ainsi on fixera les valeurs suivantes :

Indice	$T_{tracking}$	MIN_PIP
FRA40	20	0.05
EURUSD	20	0.15

• Résultats

L'exécution de l'algorithme renvoit les résultats suivant pour la variation de l'equity :

Les résultats sont assez différents pour le FRA40 et l'EURUSD. Pour le FRA40, le comportement de l'equity ressemble aux résultats que nous avons trouvés auparavant lors du backtesting opéré sur le FRA40 ; on ressort avec un rendement positif. Quant à l'EURUSD, l'algorithme n'a pas bien tourné ; au début du backtesting le modèle résiste aux éventuelles pertes avant de s'éffondrer après le tiers du temps du backtesting.

Cet effondrement du coté de l'EUR/USD est principalement dû à la méthode de raffinement que nous avons utilisée pour deux indices à la fois. Cette dernière favorise un indice par rapport à un autre, ce qui entraîne la réussite du backtesting pour l'un des deux indices uniquement. Ainsi, l'intersection de clusters ne semble pas mettre en lumière des patterns dans les réactions

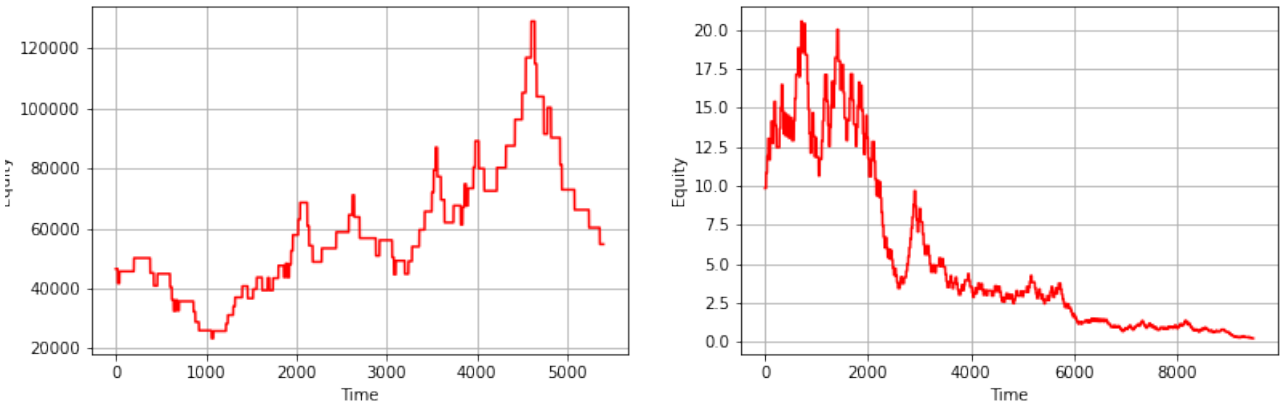


Figure 27 – Résultats du backtesting sur deux indices à la fois ; le FRA40 à gauche et l'EURUSD à droite

des investisseurs. Pour l'expliquer, les causes pourraient être nombreuses : différences de volume échangé entre les deux actifs, horaires de trading différents, profils d'investisseurs différents, ...

BIBLIOGRAPHIE

[1] F. Chollet, *L'apprentissage profond avec Python*, 2020

[2] C. Tiflin, *LSTM Recurrent Neural Networks for Signature Verification : A Novel Approach*, 2012

[3] Ikram Chraibi Kaadoud, *Apprentissage de séquences et extraction de règles de réseaux récurrents : application au traçage de schémas techniques*, 2012

[4] Alex Sherstinsky, *Fundamentals of Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM) Network*, 2018

[5] Masashi SHIBATA, *An Introduction to the Implementation of Optuna, a Hyperparameter Optimization Framework*, 2021

[6] Yenwee Lim, *State-of-the-Art Machine Learning Hyperparameter Optimization with Optuna*, 2020

[7] Kingma, D.P. and Welling, *Auto-Encoding Variational Bayes*, 2013
Alfredo Canziani, *Modèles Génératifs et Auto-encodeurs variationnels*, 2020

[8] Manuel Fritz,Michael Behringer, Holger Schwarz, *LOG-Means : Efficiently Estimating the Number of Clusters in Large Datasets*, University of Stuttgart, Germany

[9] Christoffersen, Peter, *Backtesting* (November 19, 2008)

[10] Ni, J., Zhang, C. (2005). *An Efficient Implementation of the Backtesting of Trading Strategies*

[11] M. Glantz and R. Kissell *Multi-Asset Risk Modeling : Techniques for a Global Economy in an Electronic and Algorithmic Trading Era*, 2013

[12] Christian Dose and Silvano Cincotti *Clustering of financial time series with application to index and enhanced index tracking portfolio, Physica A : Statistical Mechanics and its Applications*, Volume 355, Issue 1, 2005, Pages 145-151.

[13] Dominique Ladiray, *Classification des series temporelles - Applications à la prévision et à la désaisonnalisation*

[14] Yu Dong, *Automatic Speech Recognition - A Deep Learning Approach*, 2015

[15] Tsegamlak Terefe, Maxime Devanne, Jonathan Weber, Dereje Hailemariam, Germain Forestier, *Autoencodeur multi-tâches pour le calcul de moyennes de séries temporelles*, 2022

[16] Mostefa Sihamdi, *Détection d'anomalies dans les séries temporelles avec des auto encodeurs LSTM*, 2021

[17] Pan, Y., Chen, D., Guo, M., Cao, J., Dongarra, J. (eds) *Parallel and Distributed Processing and Applications*. ISPA 2005. Lecture Notes in Computer Science, vol 3758. Springer, Berlin, Heidelberg.